CAMPUSFOOD – SMART COLLEGE FOOD & STORE REVIEW MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

ROSHAN.K

(241401083)

in partial fulfilment of the End Semester Examination for the course

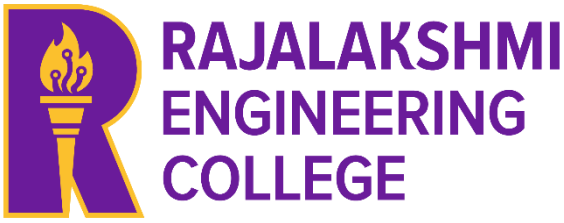
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

[Computer Science & Business System]



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

2026-2027

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "CAMPUSFOOD – SMART COLLEGE FOOD & STORE REVIEW MANAGEMENT SYSTEM" is the bonafide work of **ROSHAN.K (241401083),** who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. K. R. Sowmia, Ph.D.,
HEAD OF THE DEPARTMENT
Department of Computer Science & Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

M. L. Alphin Ezhil Manuel,
COURSE FACULTY
Assistance Professor (SG)
Department of Computer Science & Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

Submitted for the End Semester Examination (Individual Project) held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

. I, ROSHAN. K (241401083), hereby declare that the project report titled "CAMPUSFOOD – SMART COLLEGE FOOD & STORE REVIEW MANAGEMENT SYSTEM", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

SIGNATURE OF THE STUDENT
ROSHAN. K
241401083

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Mr. S. Meganathan, B.E., F.I.E., Chairperson, Dr. Thangam Meganathan, Ph.D., and Vice-Chairman, Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, M.E., Ph.D., Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. K. R. Sowmia, Ph.D., Head of the Department, Department of Computer Science and Business Systems, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Mr. M. L. Alphin Ezhil Manuel, Assistant Professor, Department of Computer Science and Business Systems, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation and the Zoho Creator platform for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

ROSHAN. K

ABSTRACT

Students on a college campus depend on several food stores for daily meals, snacks and beverages, yet the channels through which they can comment on food quality, hygiene, pricing or service are usually informal. Feedback is given verbally, on paper slips or in messaging groups, which makes it difficult to record, route to the responsible store manager, track to closure or analyse. Store managers rarely see consolidated ratings, and administrators lack a reliable view of how well complaints are being handled.

This project presents CampusFood, a Smart College Food & Store Review Management System developed on the Zoho Creator low-code platform for Rajalakshmi Engineering College, Chennai. A low-code approach suits the problem because a complete, secure and mobile-accessible data application can be built quickly with forms, reports and pages, while Deluge scripting supplies the business rules that configuration alone cannot express. The application is organised around six related forms: Student Registration, Store Management, Food Menu, Food Reviews, Complaints and Store Responses, connected through Lookup fields. Fifteen reports and a UI Builder dashboard named CampusFood Dashboard present student, store, menu, rating and complaint information, including four charts. Three roles – Administrator, Store Manager and Student – are defined with record-level access so that each user sees only the relevant data. The report also describes the proposed Deluge validations, default-status rules, form workflows and Zoho Mail notifications, and states clearly which of these items remain to be verified in the application.

A test plan of twenty test cases covering functional, validation and role-based access scenarios is prepared to verify each requirement, and the deployment, security and governance practices suitable for a Zoho Creator application are described. The expected benefit is a single traceable channel through which feedback is captured, answered and monitored, replacing scattered manual communication. Measured results are to be recorded against the test plan during evaluation.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Role-Based Access Control; Campus Food Management; Student Feedback

PROJECT AT A GLANCE

Attribute	Details
Project Title	CampusFood – Smart College Food & Store Review Management System
Application Domain	Education – College Food Store Management and Student Feedback System
Student Name & Register No.	ROSHAN. K - 241401083
Zoho Creator Edition	Trial Edition
Application Name (Link Name)	Campus Food Management System
Published / Live URL	https://creatorapp.zoho.in/roshan.k.2024.csbs_rajala553/campus-food-management-system1#Section_1
Demo Credentials (per role)	Admin, CEO, Store Manager , Students
Number of Forms / Subforms	6 (Student Registration, Store Management, Food Menu, Food Reviews, Complaints, Store Responses)
Number of Reports (by type)	6 in total Breakdown by type: List: 6
Number of Pages / Dashboards	1 (Campus Food Management Dashboard, built with UI Builder)
Number of Workflows	3 – Food Reviews, Complaints, Food Menu and Store Responses workflows
Deluge Functions / Custom Actions	Deluge scripts used in workflow automation and form validation – confirm
Integrations	Zoho Mail notifications (proposed; implementation status to be confirmed)
Roles & Profiles Defined	Zoho Creator automated email notification for complaint submissions (configured and tested)
Environments Used	Administrator, Store Manager, Student (design; configuration status to be confirmed)
Date of Production Deployment	

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES viii
LIST OF FIGURES ix
LIST OF ABBREVIATIONS x
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 5
2.1 Study of the Existing System 5
2.2 Limitations of the Existing System 5
2.3 Low-Code versus Traditional Development 6
2.4 Literature Survey 7
2.5 Emerging Trends in Low-Code Development 8
2.6 Proposed System 8
REQUIREMENT ANALYSIS 10
3.1 Stakeholders and User Roles 10
3.2 Functional Requirements 10
3.3 Non-Functional Requirements 12
3.4 Hardware and Software Requirements 12
3.5 Use Case Model 12
SYSTEM DESIGN 15
4.1 System Architecture 15
4.2 Data Model 15
4.3 Form Design 16
4.4 Process and Workflow Design 20
4.5 User Interface and Navigation Design 20
4.6 Access Control Design 21
IMPLEMENTATION 23
5.1 Application Setup 23
5.2 Forms and Data Capture 24
5.3 Reports 25

5.4 Pages and Dashboards 27
5.5 Deluge Scripting 28
5.6 Workflows and Automation 30
5.7 Integrations 32
5.8 User Management and Role-Based Access 34
5.9 AI and Analytics Features 35
5.10 Pro-Code Extension 36
5.11 Mobile and Portal Access 36
TESTING AND DEBUGGING 38
6.1 Testing Strategy 38
6.2 Test Cases 38
6.3 Debugging Deluge Scripts 40
6.4 Defect Log 41
6.5 User Acceptance Testing 41
DEPLOYMENT, SECURITY AND GOVERNANCE 42
7.1 Environments and Deployment Strategy 42
7.2 Performance Optimisation 43
7.3 Security Features 43
7.4 Governance and Administration 44
RESULTS AND DISCUSSION 45
8.1 Application Output 45
8.2 Achievement of Objectives 46
8.3 Comparison with the Existing System 47
8.4 Limitations of the Low-Code Approach 47
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 48
CONCLUSION AND FUTURE ENHANCEMENTS 49
9.1 Conclusion 49
9.2 Future Enhancements 49
9.3 Learning Outcomes and Career Relevance 49
REFERENCES 51
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 52
APPENDIX B – COURSE OUTCOME MAPPING 54
APPENDIX C – COMPLETE DELUGE SOURCE CODE 55
APPENDIX D – USER MANUAL 58
APPENDIX E – EVALUATION RUBRIC 60
APPENDIX F – VIVA-VOCE RECORD SHEET 62

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 6
Table 2.2: Summary of Literature Survey 7
Table 3.1: Stakeholders and User Roles 10
Table 3.2: Functional Requirements 10
Table 3.3: Non-Functional Requirements 12
Table 3.4: Hardware and Software Requirements 12
Table 3.5: Use Case Description – Submit Food Review 13
Table 3.6: Use Case Description – Register Complaint and Receive Store Response 13
Table 3.7: Use Case Description – Manage Food Menu 14
Table 4.1: Entities and Relationships 16
Table 4.2: Field Specification – Student Registration Form 17
Table 4.3: Field Specification – Store Management Form 17
Table 4.4: Field Specification – Food Menu Form 18
Table 4.5: Field Specification – Food Reviews Form 18
Table 4.6: Field Specification – Complaints Form 19
Table 4.7: Field Specification – Store Responses Form 19
Table 4.8: Role–Permission Matrix 21
Table 5.1: Reports Implemented 25
Table 5.2: Deluge Scripts Implemented 28
Table 5.3: Workflows Implemented 30
Table 5.4: Integrations Implemented 32
Table 6.1: Test Cases 38
Table 6.2: Defect Log 41
Table 6.3: User Acceptance Feedback 41
Table 7.1: Deployment Checklist 42
Table 7.2: Performance Optimisation Results 43
Table 7.3: Security Controls Implemented 43
Table 8.1: Mapping of Objectives to Outcomes 46
Table 8.2: Existing System versus Proposed Application 47
Table A.1: Feature Coverage Checklist 52
Table B.1: Mapping of Course Outcomes to Report Chapters 54
Table E.1: Evaluation Rubric for the Individual Project 60
Table E.2: Performance Level Descriptors 60
Table F.1: Viva-Voce Questions and Responses 62
Table F.2: Marks Summary 62

LIST OF FIGURES

Figures 2.1 to 4.3 contain diagrams prepared for this project. The remaining figures have a reserved space for the student's own screenshots, preceded by a text description of what each should show.

Figure 2.1: As-Is Process Flow of the Existing System 5
Figure 3.1: Use Case Diagram of the Application 13
Figure 4.1: System Architecture of the Application 15
Figure 4.2: Entity–Relationship Diagram 16
Figure 4.3: Workflow / Process Flow for Complaint Handling 20
Figure 4.4: Navigation Structure and Wireframe of the CampusFood Dashboard 21
Figure 5.1: Application Home and Navigation Sections 23
Figure 5.2: Food Reviews Form in the Form Builder 24
Figure 5.3: Food Reviews Form in Live Mode 25
Figure 5.4: Open Complaints Report 27
Figure 5.5: CampusFood Dashboard 28
Figure 5.6: Complaint Workflow Configuration 32
Figure 5.7: Connection Setup and Response Output 34
Figure 5.8: Roles and Permission Profiles Configuration 35
Figure 5.9: Analytics Features in Use 36
Figure 5.10: Application on Mobile 37
Figure 6.1: Deluge Debug Output / Log 41
Figure 7.1: Environments / Publish to Production 42
Figure 7.2: Audit Trail / Administration Settings 44
Figure 8.1: Scenario Step 1 – Student Registers and Submits a Review 45
Figure 8.2: Scenario Step 2 – Student Registers a Complaint 46
Figure 8.3: Scenario Step 3 – Store Manager Responds and the Complaint Is Resolved 46

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CO	Course Outcome
DS	Deluge Script (script identifier)
ER	Entity–Relationship
FR	Functional Requirement
JSON	JavaScript Object Notation
NFR	Non-Functional Requirement
OAuth	Open Authorization
RBAC	Role-Based Access Control
REST	Representational State Transfer
TC	Test Case
UC	Use Case
UI	User Interface
URL	Uniform Resource Locator
Zia	Zoho’s AI assistant

CHAPTER 1

INTRODUCTION

1.1 Background

A residential engineering college campus is a small community in which thousands of students depend every day on a handful of food outlets such as canteens, tiffin counters, juice bars and bakeries. The quality, hygiene, pricing and availability of food at these outlets directly affect student health, satisfaction and, indirectly, academic performance. Rajalakshmi Engineering College, Chennai, is a large autonomous institution where food stores operate throughout the day for students, faculty and staff.

In most campuses the process of collecting opinions about these stores is informal. A student who is dissatisfied with a meal may speak to the counter staff, write on a paper suggestion slip or post a comment in a messaging group. Store managers, who are responsible for the menu and for service quality, seldom receive these opinions in an organised form, and the college administration has no consolidated view of how each store is performing. The people involved in the process are the students who consume the food, the store managers who run each outlet, and the administrators who supervise the stores on behalf of the college.

This project, titled CampusFood – Smart College Food & Store Review Management System, addresses this gap by building a centralised application on the Zoho Creator low-code platform. The application records the stores, their menus, student reviews, complaints and the responses given by store managers, and it presents the information through role-specific reports and a dashboard.

1.2 Problem Statement

Colleges lack a single, structured and traceable channel through which students can view food stores and menus, rate and review the food, register complaints, and receive responses from the store managers, while the administration monitors the overall situation. As a result, feedback is scattered, complaints are not tracked to closure, and decisions about stores are made without reliable evidence. The specific pain points are listed below.

- Student opinions are collected through verbal remarks, paper slips or messaging groups, so no permanent and searchable record of ratings and reviews exists.
- Complaints about food quality, hygiene, pricing or service are not routed to the responsible store manager in a consistent manner, and students receive no acknowledgement or resolution status.
- Store managers have no consolidated view of the ratings and complaints relating to their store, and the administration cannot compare stores objectively.
- Data and access are uncontrolled: any person who holds a shared sheet or group can read or alter the information, and there is no audit of who changed what.

1.3 Objectives

The objectives of the project are stated below. Each objective is traced to the implementation modules in Chapter 5 and to the results in Chapter 8 (Table 8.1).

- To design a normalised data model of six forms – Student Registration, Store Management, Food Menu, Food Reviews, Complaints and Store Responses – connected through Lookup fields.
- To automate data validation and default behaviour (unique register numbers, valid store timings, non-negative prices, ratings between 1 and 5, default review and complaint statuses) using Deluge scripting and form workflows.
- To provide role-based access for the Administrator, Store Manager and Student roles, with record-level restrictions so that a manager sees only the assigned store and a student sees only personal records.
- To provide fifteen reports and a UI Builder dashboard named CampusFood Dashboard that display key indicators and charts on stores, menus, ratings and complaints.
- To plan the integration of the application with Zoho Mail for notifying managers and students, using connections and OAuth authorisation, and to describe error handling for these integrations.

1.4 Scope of the Project

The scope of the project is limited to the collection and management of feedback and store information. The items inside and outside the scope are as follows.

In scope:

- Registration of students and maintenance of food store details.
- Maintenance of the food menu of each store, including category, price and availability.
- Submission and moderation of food reviews with ratings.
- Registration, tracking and resolution of complaints, and submission of store responses.
- Three user roles – Administrator, Store Manager and Student – accessed through the Zoho Creator web interface. Access through the Zoho Creator mobile app or a customer portal is considered in Section 5.11 and is subject to verification.

Out of scope:

- Online food ordering, payment processing, billing, delivery and inventory management; the application does not handle food orders.
- Integration with the college ERP or student information system; students are registered within the application itself.
- Custom pro-code components, which are discussed only as future enhancements (Section 5.10).

1.5 Justification for a Low-Code Approach

Low-code application platforms allow business applications to be built through visual modelling, pre-built components and limited scripting, instead of fully hand-written code [1], [2]. For CampusFood this approach is appropriate for several reasons. First, the application is a data-centred system made of forms, reports and approval-like status changes, which is precisely the class of application that Zoho Creator supports through its form builder, report types and page builder [2]. Second, the development effort is small enough to be completed by a single student developer within the semester, and the resulting application can later be maintained by a citizen developer in the college administration without specialised programming skills. Third, Zoho Creator provides hosting, backup, user management, role-based security and access through browsers and mobile devices as built-in services, so the project does not require separate infrastructure. Finally, Deluge scripting offers a controlled way of adding the few business rules that configuration alone cannot express, such as duplicate checks and

conditional notifications [2]. The trade-offs of the approach, including dependence on the vendor [3], are discussed in Section 8.4.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements. The references and Appendices A to F follow, containing the feature coverage checklist, course outcome mapping, Deluge source code, user manual, evaluation rubric and viva-voce record sheet.

CHAPTER 2

LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The existing arrangement for handling food-related feedback in a typical college campus is manual and fragmented.

A student who consumes food at a store and is dissatisfied usually raises the matter orally with the counter staff, drops a written slip in a suggestion box, or complains to friends or in a class group. The counter staff may or may not inform the store manager. When the manager is informed, the response is informal and is rarely recorded. The student is generally not told whether any action was taken. Menu items and prices are displayed on a notice board at the store and are updated by hand, so students cannot check them in advance. The college administration comes to know about problems only when a serious complaint is escalated, and ratings or satisfaction levels of different stores are never compared on a common basis.

The as-is process flow proceeds in sequence as follows. Step 1: the student buys and consumes food at a campus store. Step 2: the student forms an opinion about quality, hygiene, price or service. Step 3: the student communicates the opinion verbally to the counter staff, through a paper suggestion slip or through an informal group message. Step 4: the counter staff or a volunteer may pass the remark to the store manager, but there is no fixed channel. Step 5: the manager decides informally whether to act, and no record of the decision is kept. Step 6: the student is usually not informed of the outcome. Step 7: the administration learns of recurring problems only through escalation by senior persons. The flow therefore has no recorded start, no tracked intermediate status and no defined end.

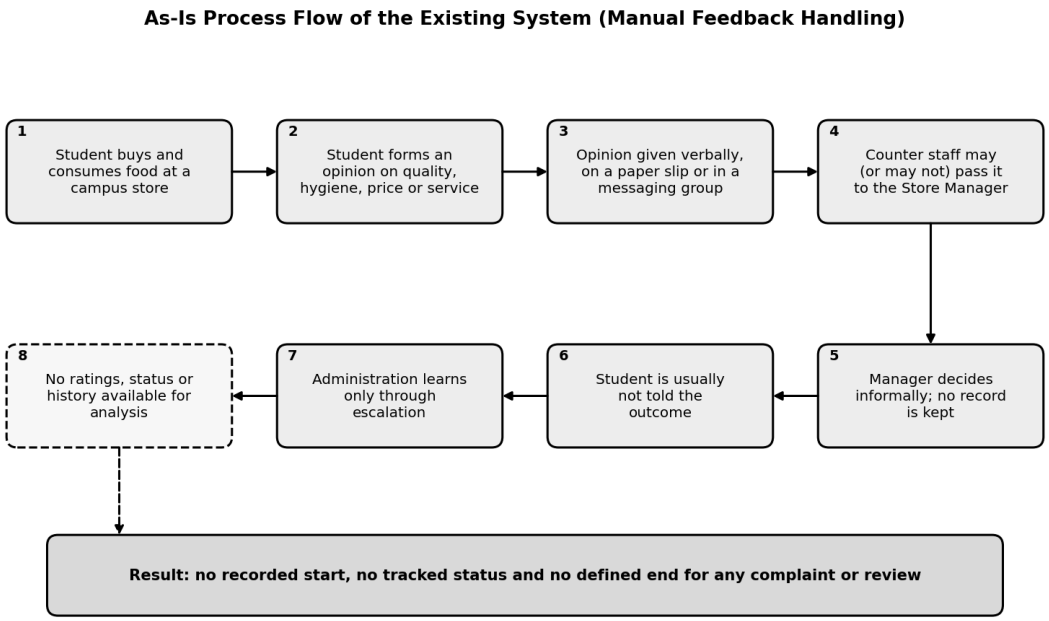


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- No permanent record: opinions given verbally or in chat groups cannot be searched, counted or analysed.
- No tracking: a complaint has no status, owner or resolution comment, so the student cannot know whether it was addressed.
- Delayed or missing communication: the store manager may not learn of a complaint at all, and there is no notification mechanism.
- No objective comparison: ratings are not collected on a uniform scale, so the administration cannot compare stores or detect declining quality.
- No access control or audit: manual sheets and group messages can be read or altered by anyone and provide no history of changes.

2.3 Low-Code versus Traditional Development

Table 2.1 compares traditional pro-code development with low-code development on Zoho Creator for the CampusFood application. The traditional-development figures are planning estimates for an application of this size and are not measurements. The low-code column records the actual effort that the student must enter from the project log. General discussions of the comparison are available in [2] and [3].

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Estimated 8–10 weeks for design, database, back end, front end and testing (planning estimate).	Estimated 2–3 weeks for six forms, reports and a dashboard (planning estimate). Actual effort
Skills required	Web front end, server-side language, relational database design, authentication and deployment skills.	Data modelling, form and report configuration, and basic Deluge scripting.
Hosting & infrastructure	Server or cloud hosting, database administration, backups and security patches to be arranged.	Hosting, database, backups and platform security are provided by Zoho Creator.
Mobile application	Separate mobile development or a responsive front end is required.	Applications are accessible through the Zoho Creator mobile app without separate development
Security & access control	Authentication, authorisation and session management must be implemented and tested.	Built-in users, roles, permission profiles and record-level criteria.
Maintenance & change	Changes require code modification, rebuilding and redeployment by a developer.	Forms, fields and reports can be modified by a citizen developer in the builder.
Customisation limits	Virtually unlimited, at the cost of effort.	Limited to platform features, with Deluge and integrations as extension points.

2.4 Literature Survey

Table 2.2 summarises the sources reviewed. They comprise course reference material, peer-reviewed articles on low-code platforms and the official Zoho documentation. The bibliographic details are given in the References section and should be re-verified against the original publications before submission; entries whose details could not be confirmed are marked for verification.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	W. Rosam and O. Kalnichevski, BPB Publications 2020	Low-Code Development: A Practical Guide	Course reference text presenting the concepts and practice of low-code development.	Source of the concepts used in Chapters 1 and 2 and in the spectrum discussion of Section 8.5.
2	Zoho Corporation (accessed oct, 2026)	Zoho Creator Help Documentation	Documents forms, reports, pages, workflows, users and permissions, and deployment in Zoho Creator.	Primary technical basis for the design and implementation of CampusFood.

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
3	A. C. Bock and U. Frank, 2021 [3]	Low-Code Platform	Characterises low-code platforms and discusses their benefits and limitations, including dependence on the platform vendor.	Supports the justification in Section 1.5 and the limitations in Section 8.4.
5	R. Sanchis, Ó. García-Perales, F. Fraile and R. Poler, 2020 [5]	Low-Code as Enabler of Digital Transformation in Manufacturing Industry	Discusses low-code as a means for faster development of business applications by users close to the process.	Supports the choice of low-code for a domain-specific process managed by non-specialist maintainers.
6	D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi and M. Wimmer, 2022 [6]	Low-code development and model-driven engineering: Two sides of the same coin?	Examines the relationship between low-code platforms and model-driven engineering and identifies open challenges.	Relates to the model-first design of the six-form data model in Chapter 4.
7	M. Waszkowski, 2019 [7]	Low-code platform for automating business processes in manufacturing	Reports the use of a low-code platform to automate business processes in a manufacturing context.	Supports the use of status-driven workflow automation for complaints and responses.

2.5 Emerging Trends in Low-Code Development

Several trends in low-code development are relevant to CampusFood. Analyst publications on low-code application platforms, such as the Gartner Magic Quadrant [8], describe a continuing shift toward platforms that combine visual development, integration and artificial intelligence. Citizen development is the first trend: staff who understand the process but are not professional programmers build and maintain applications themselves [5]. CampusFood follows this trend by keeping its data model small and its rules readable, so that a college administrator can later add a store type or a complaint category without developer support.

The second trend is AI-assisted development and AI-enabled applications. Zoho Creator offers an assistant, Zia, and AI features such as sentiment analysis. These could be applied to the Review Comments field to classify student feedback automatically, and this is recorded as a future enhancement in Section 9.2 rather than as an implemented feature. The third trend is hyper-automation, in which workflows, notifications and integrations reduce manual hand-offs; the proposed complaint notifications through Zoho Mail apply this idea at a modest scale. The fourth trend is composability, in which small applications exchange data through APIs [6]; the separation of stores, students and feedback into related forms keeps CampusFood open to such reuse.

2.6 Proposed System

The proposed system is the CampusFood application on Zoho Creator. It replaces informal feedback channels with six related forms and gives each stakeholder a defined view of the data. The way in which the proposed system overcomes each limitation listed in Section 2.2 is summarised below.

- **No permanent record** is overcome by storing every student, store, menu item, review, complaint and response as a record in the Student Registration, Store Management, Food Menu, Food Reviews, Complaints and Store Responses forms.
- **No tracking** is overcome by the Complaint Status and Response Status fields, with a default Open status for each new complaint and a Resolution Comments field for the outcome.
- **Delayed communication** is addressed by the Store Responses form, which links a response to the relevant complaint, and by the proposed Zoho Mail notifications (Section 5.7).
- **No objective comparison** is overcome by the numeric Rating field, the Reviews by Store and Reviews by Rating reports, and the dashboard charts.
- **Outdated menu information** is overcome by the Food Menu form, whose Availability field feeds the Available Food Items and Unavailable Food Items reports.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Three user roles are identified for the application. Table 3.1 describes each role and its responsibilities. The roles describe the intended design; the configuration of the corresponding permission profiles is described and its verification status is stated in Section 5.8.

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	College staff member who supervises the food stores and the application.	Manage all records (students, stores, menus, reviews, complaints, responses); monitor reports and the CampusFood Dashboard; manage permissions and application settings.
Store Manager	Person in charge of one food store, identified through the Manager Name and Manager Email fields of the Store Management form.	Manage the food menu of the assigned store; view reviews and complaints of that store; submit responses to complaints; update complaint status where permitted.
Student	Registered student of the college who uses the food stores.	View permitted store and menu information; submit food reviews; register complaints; track own complaint status and the responses received.

3.2 Functional Requirements

The functional requirements are listed in Table 3.2. Each requirement is testable and is referred to by its FR identifier in the design tables of Chapter 4, the implementation tables of Chapter 5 and the test cases of Table 6.1.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow a student to be registered with Student Name, Register Number, Email, Department, Year, Phone Number and Registration Date, with the mandatory details enforced.	Administrator, Student	High
FR-02	The system shall prevent two students from being registered with the same Register Number.	Administrator, Student	High
FR-03	The system shall allow food stores to be recorded with Store Name, Store Type, Location, Manager Name, Manager Email, Opening Time, Closing Time and Store Status.	Administrator	High
FR-04	The system shall validate store details and shall reject a Closing Time that is not later than the Opening Time.	Administrator	Medium

FR ID	Requirement	Role(s)	Priority
FR-05	The system shall allow food menu items to be recorded against a store with Item Name, Category, Description, Price and Availability (and an optional Food Image).	Administrator, Store Manager	High
FR-06	The system shall reject a negative Price for a menu item.	Administrator, Store Manager	High
FR-07	The system shall allow students to view permitted store details and the menu items of each store.	Student	High
FR-08	The system shall allow a student to submit a food review for a store with Rating and Review Comments.	Student	High
FR-09	The system shall accept only Rating values between 1 and 5.	Student	High
FR-10	The system shall assign a default Review Status (Pending) to every new review, and the Administrator shall be able to change it to Published.	Administrator	Medium
FR-11	The system shall allow a student to register a complaint about a store with Complaint Type and Description.	Student	High
FR-12	The system shall assign the default Complaint Status Open to every new complaint.	Student, Administrator	High
FR-13	The system shall allow a Store Manager to view only the reviews and complaints that relate to the assigned store.	Store Manager	High
FR-14	The system shall allow a Store Manager to submit a Store Response linked to the relevant complaint and store, and shall validate the response information.	Store Manager	High
FR-15	The system shall allow the Complaint Status and Resolution Comments to be updated by permitted roles, and the status to be updated when a response is submitted or the complaint is resolved.	Store Manager, Administrator	Medium
FR-16	The system shall allow a student to track the status of own complaints and the responses given.	Student	High
FR-17	The system shall provide the fifteen reports listed in Table 5.1.	All roles (as permitted)	Medium
FR-18	The system shall provide the CampusFood Dashboard showing Total Students, Total Stores, Total Menu Items, Average Food Rating, Open Complaints and Total Store Responses, with four charts.	Administrator	Medium
FR-19	The system shall enforce role-based access with record-level restrictions for the Administrator, Store Manager and Student roles.	All roles	High
FR-20	The system shall notify the Store Manager when a complaint is registered, and the student when a response is submitted or the complaint is resolved (proposed through Zoho Mail; see Section 5.7).	Store Manager, Student	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports with up to 1,000 records shall load within 5 seconds under normal network conditions (design target; to be measured and recorded in Table 7.2).
NFR-02	Security	Users shall access only the data permitted by their role, and a student shall not view the records of another student.
NFR-03	Usability	The application shall be usable in a web browser and, where verified, in the Zoho Creator mobile app, with forms that a first-time student can complete without training.
NFR-04	Availability	The application shall be available whenever the Zoho Creator service is available, since hosting is provided by the platform.
NFR-05	Maintainability	Lists of values such as store types, categories and complaint types shall be modifiable in the form builder without scripting changes.
NFR-06	Auditability	Changes to records shall be traceable through the platform audit trail and the Added User and Added Time information of each record.
NFR-07	Data integrity	Relationships between forms shall be maintained through Lookup fields so that every review, complaint and response refers to an existing student, store or complaint.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator [Trial version]
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome, Microsoft Edge or Mozilla Firefox (latest version)
API testing tool	Postman (required only if the Zoho Mail API integration proposed in Section 5.7 is implemented)
Other Zoho services	Zoho Mail (proposed for notifications)
Mobile	Zoho Creator mobile app for Android or iOS
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

The use case model identifies three actors – Administrator, Store Manager and Student – and the main functions available to each. Figure 3.1 describes the use case diagram in text, and Tables 3.5 to 3.7 describe three key use cases in detail.

The Student actor is associated with the use cases Register as Student, View Stores and Menu, Submit Food Review, Register Complaint and Track Complaint Status and

Responses. The Store Manager actor is associated with Manage Food Menu, View Store Reviews, View Store Complaints, Submit Store Response and Update Complaint Status. The Administrator actor is associated with Manage Students, Manage Stores, Manage Food Menu, Moderate Reviews, Manage Complaints and Responses, View Reports and Dashboard, and Manage Permissions and Settings. The use case Submit Store Response includes the use case View Store Complaints, because a manager responds to a complaint that has been viewed. The use case Register Complaint is followed by Submit Store Response and Track Complaint Status and Responses, forming the complaint life cycle shared by the Student and the Store Manager.

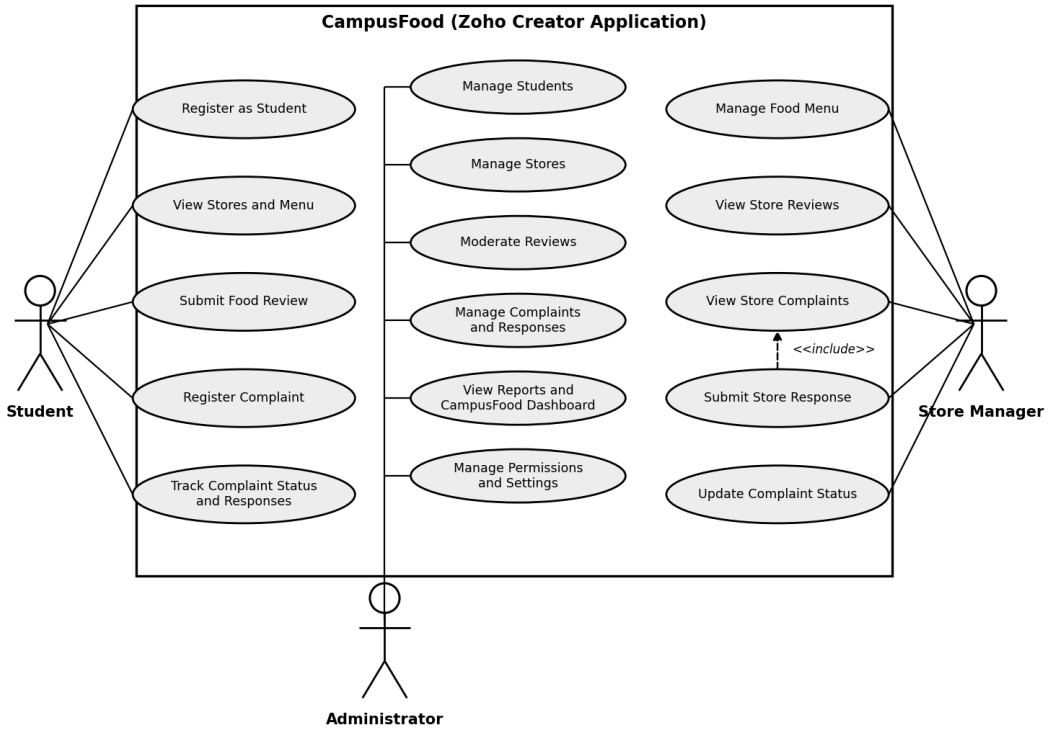


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Submit Food Review

Use Case ID	UC-01
Use Case Name	Submit Food Review
Actor(s)	Student
Pre-condition	The student is registered and logged in; the store exists in the Store Management form with an active status.

Main Flow	1. The student opens the Food Reviews form. 2. The student selects the Student and Store through the Lookup fields. 3. The student enters the Rating (1 to 5) and Review Comments. 4. The student submits the form; the system validates the rating and sets the Review Status to Pending. 5. The system displays the success message.
Alternate Flow	If the Rating is outside 1 to 5 or a mandatory field is empty, the system shows a validation message and does not save the record.
Post-condition	A review record with status Pending exists and appears in the Pending Reviews report until the Administrator publishes it.
Related FR	FR-08, FR-09, FR-10

Table 3.6: Use Case Description – Register Complaint and Receive Store Response

Use Case ID	UC-02
Use Case Name	Register Complaint and Receive Store Response
Actor(s)	Student, Store Manager
Pre-condition	The student is registered and logged in; the Store Manager has an account and is assigned to the store concerned.
Main Flow	1. The student opens the Complaints form and selects the Student and Store. 2. The student chooses the Complaint Type and enters the Description. 3. On submission the system sets the Complaint Status to Open. 4. The Store Manager opens the Open Complaints report and selects the complaint. 5. The manager submits a Store Responses record linked to the complaint, with the Response Message. 6. The complaint status is updated and the student views the response and status in the Complaints and Store Responses reports.
Alternate Flow	If the Response Message is empty, the system rejects the submission. If email notification is implemented, the manager and student receive notifications at steps 3 and 5.
Post-condition	The complaint has an associated response and an updated status; the Responses by Store report includes the new response.
Related FR	FR-11, FR-12, FR-13, FR-14, FR-15, FR-16, FR-20

Table 3.7: Use Case Description – Manage Food Menu

Use Case ID	UC-03
Use Case Name	Manage Food Menu
Actor(s)	Store Manager, Administrator
Pre-condition	The user is logged in; the store exists in the Store Management form; for a Store Manager, the store is the assigned store.

Main Flow	1. The user opens the Food Menu form. 2. The user selects the Store, and enters the Item Name, Category, Description, Price and Availability. 3. The system validates that the Price is not negative. 4. The record is saved and appears in the Food Menu Report and in the Available Food Items or Unavailable Food Items report according to its Availability.
Alternate Flow	If the Price is negative, the system shows a message and does not save. To mark an item sold out, the user edits the record and changes Availability.
Post-condition	The menu of the store reflects the current items, prices and availability, and students can view them.
Related FR	FR-05, FR-06, FR-07

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

CampusFood follows the layered structure of a Zoho Creator application. Figure 4.1 describes the layers in text, and the explanation below states the role of each layer.

The architecture has five layers arranged from the user to the external services. The access layer consists of the three user groups – Administrator, Store Manager and Student – who reach the application through a web browser and, where verified, the Zoho Creator mobile app. The presentation layer consists of the application menu, the six forms, the fifteen reports and the CampusFood Dashboard built with UI Builder. The logic layer consists of form validation rules, Deluge scripts and form workflows that enforce business rules and set default values. The data layer consists of the records of the six forms, connected through Lookup fields, and the platform audit information. The integration layer consists of the Zoho Mail notifications proposed for managers and students, which use Zoho Creator connections and OAuth authorisation if implemented. Role-based permission profiles cut across the presentation, logic and data layers and decide which components and records each user can reach.

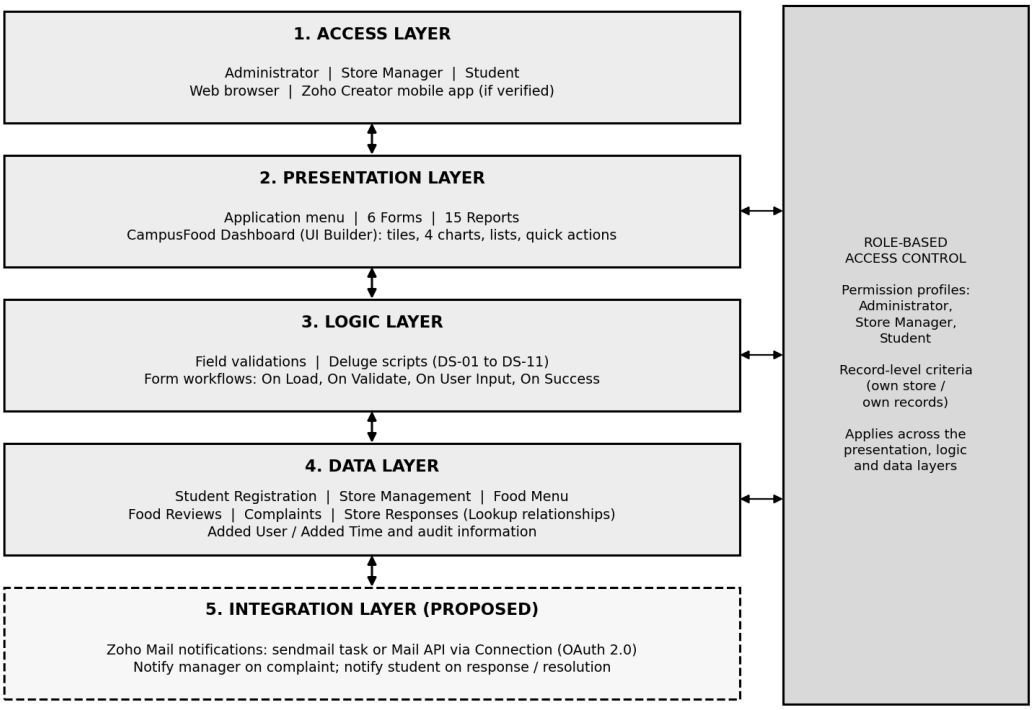


Figure 4.1: System Architecture of the Application

The access layer separates users by role, so that the same application shows different menus and records to an administrator, a store manager and a student. The presentation layer lets users enter and view data without writing code. The logic layer contains the Deluge scripts described in Section 5.5; Deluge is the scripting language of Zoho Creator and runs on form events such as On Load, On Validate and On Success [4]. The data layer is managed by the platform, so no database administration is needed. The integration layer is kept separate because it depends on external authorisation, and its status is explained in Section 5.7.

4.2 Data Model

The data model contains six entities, each implemented as a Zoho Creator form. Student Registration and Store Management are the two master forms. Food Menu, Food Reviews, Complaints and Store Responses are transaction forms that refer to the master forms, or to Complaints, through Lookup fields. In Zoho Creator a Lookup field stores a reference to a record of another form and realises a one-to-many relationship between the parent and the child form. No Subform is used, because none of the six forms contains a repeating group of rows that belongs to a single parent record.

The entity–relationship structure is described as follows. Store Management is the central entity: one store has many menu items (Food Menu), many reviews (Food Reviews), many complaints (Complaints) and many store responses (Store Responses). Student Registration is related to Food Reviews and to Complaints: one student can submit many reviews and many complaints, and each review or complaint belongs to exactly one student. Complaints is related to Store Responses: one complaint can receive one or more responses, and each response refers to exactly one complaint. Each child form therefore carries one or two Lookup fields pointing to its parents, and there are no many-to-many relationships between the forms.

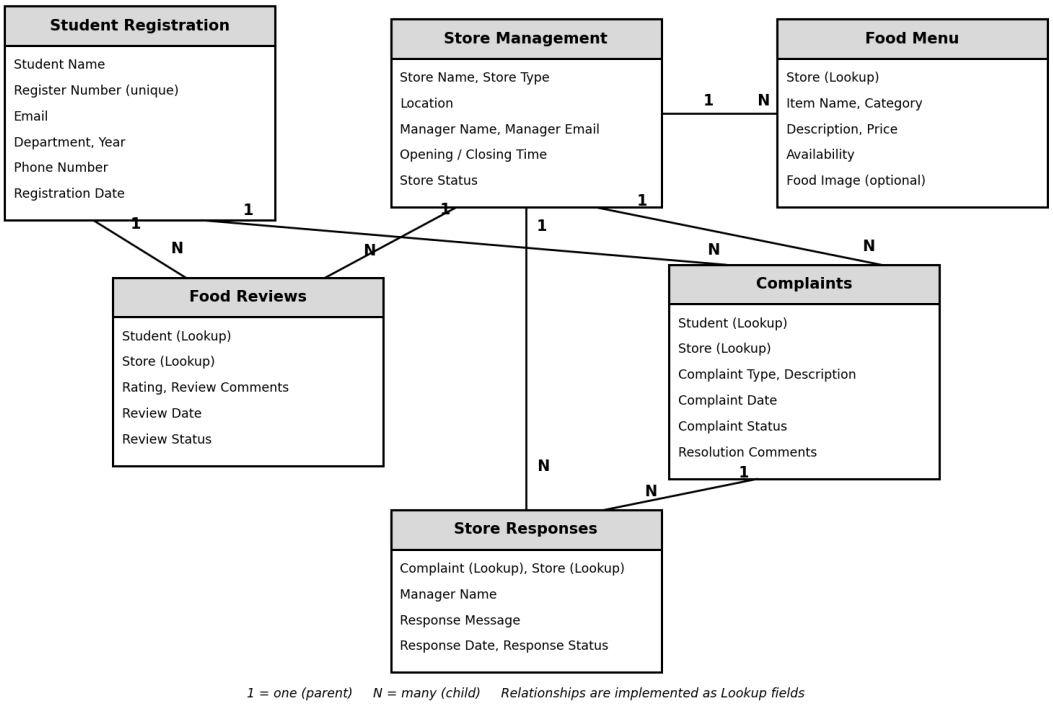


Figure 4.2: Entity–Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Food Menu	Item Name, Category, Price, Availability	Store Management	Many-to-One (many items per store)	Lookup field Store in Food Menu
Food Reviews	Rating, Review Comments, Review Status	Student Registration	Many-to-One (many reviews per student)	Lookup field Student in Food Reviews
Food Reviews	Rating, Review Date	Store Management	Many-to-One (many reviews per store)	Lookup field Store in Food Reviews
Complaints	Complaint Type, Complaint Status	Student Registration	Many-to-One (many complaints per student)	Lookup field Student in Complaints

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Complaints	Complaint Type, Complaint Date	Store Management	Many-to-One (many complaints per store)	Lookup field Store in Complaints
Store Responses	Response Message, Response Status	Complaints	Many-to-One (one or more responses per complaint)	Lookup field Complaint in Store Responses
Store Responses	Manager Name, Response Date	Store Management	Many-to-One (many responses per store)	Lookup field Store in Store Responses

4.3 Form Design

This section gives the field specification of each of the six forms in Tables 4.2 to 4.7. The field labels are those of the application. The field types, link names, validations and drop-down values are the design specification derived from the field list; they follow the default Zoho Creator convention in which the link name is the label with underscores, and they must be verified against the form builder before submission .Zoho Creator also adds the system fields Added User and Added Time to every record.

Table 4.2: Field Specification – Student Registration Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Student Name	Student_Name	Single Line	Yes	Not empty	Full name of the student
Register Number	Register_Number	Single Line	Yes	No duplicate values (DS-01)	Unique identifier of the student
Email	Email	Email	Yes	Valid email format	Used to identify the student for record-level access
Department	Department	Drop Down	Yes	Choice list	Department of study
Year	Year	Drop Down	Yes	I / II / III / IV	Year of study
Phone Number	Phone_Number	Phone	Yes	Valid phone format	Contact number
Registration Date	Registration_Date	Date	No	Default: current date	Date of registration

Table 4.3: Field Specification – Store Management Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Store Name	Store_Name	Single Line	Yes	Not empty	Name of the food store
Store Type	Store_Type	Drop Down	Yes	Choice list [e.g., Canteen / Juice Bar / Bakery]	Kind of outlet
Location	Location	Single Line	Yes	Not empty	Place on the campus

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Manager Name	Manager_Name	Single Line	Yes	Not empty	Name of the store manager
Manager Email	Manager_Email	Email	Yes	Valid email format	Used for notifications and store-level access
Opening Time	Opening_Time	Date-Time or time entry	Yes	Must precede Closing Time (DS-02)	Start of daily operation
Closing Time	Closing_Time	Date-Time or time entry	Yes	Must be later than Opening Time (DS-02)	End of daily operation
Store Status	Store_Status	Drop Down	Yes	Active / Inactive	Whether the store is operating

Table 4.4: Field Specification – Food Menu Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Store	Store	Lookup (Store Management)	Yes	Selects an existing store	Relationship to Store Management
Item Name	Item_Name	Single Line	Yes	Not empty	Name of the food item
Category	Category	Drop Down	Yes	Choice list [e.g., Breakfast / Meals / Snacks / Beverages – confirm]	Used by the Food Items by Category chart
Description	Description	Multi Line	No	—	Short description of the item
Price	Price	Currency or Decimal	Yes	Must not be negative (DS-03)	Price in rupees
Availability	Availability	Drop Down or Radio	Yes	Available	Drives the availability reports
Food Image	Food_Image	Image	No (optional)	—	Optional; no image is included in this report

Table 4.5: Field Specification – Food Reviews Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Student	Student	Lookup (Student Registration)	Yes	Selects an existing student	Relationship to Student Registration
Store	Store	Lookup (Store Management)	Yes	Selects an existing store	Relationship to Store Management
Rating	Rating	Number or Drop Down 1–5	Yes	Between 1 and 5 (DS-04)	Basis of the average rating
Review Comments	Review_Comments	Multi Line	No	—	Free-text opinion of the student

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Review Date	Review_Date	Date	No	Default: current date (DS-05)	Date of the review
Review Status	Review_Status	Drop Down	Yes	Default: Pending (DS-05); Pending / Published	Drives Pending Reviews and Published Reviews

Table 4.6: Field Specification – Complaints Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Student	Student	Lookup (Student Registration)	Yes	Selects an existing student	Relationship to Student Registration
Store	Store	Lookup (Store Management)	Yes	Selects an existing store	Relationship to Store Management
Complaint Type	Complaint_Type	Drop Down	Yes	Choice list [e.g., Food Quality / Hygiene / Pricing / Service – confirm]	Classification of the complaint
Description	Description	Multi Line	Yes	Not empty (DS-06)	Details of the complaint
Complaint Date	Complaint_Date	Date	No	Default: current date (DS-06)	Date of registration
Complaint Status	Complaint_Status	Drop Down	Yes	Default: Open (DS-06); Open / In Progress / Resolved	Drives the status reports and the dashboard
Resolution Comments	Resolution_Comments	Multi Line	No	—	Outcome recorded when resolved

Table 4.7: Field Specification – Store Responses Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Complaint	Complaint	Lookup (Complaints)	Yes	Selects an existing complaint	Relationship to Complaints
Store	Store	Lookup (Store Management)	Yes	Filled from the selected complaint (DS-08)	Relationship to Store Management
Manager Name	Manager_Name	Single Line	Yes	Not empty	Name of the responding manager
Response Message	Response_Message	Multi Line	Yes	Not empty (DS-09)	Reply given to the student
Response Date	Response_Date	Date	No	Default: current date	Date of the response
Response Status	Response_Status	Drop Down	Yes	Choice list [e.g., Submitted / Closed – confirm]	State of the response

4.4 Process and Workflow Design

Two processes are automated or supported by the application: the review life cycle and the complaint life cycle. Figure 4.3 describes the complaint life cycle, which is the principal process, and the review life cycle is summarised after it. Each step indicates where a validation, a Deluge script or a workflow is intended to run.

Step 1: the student opens the Complaints form and, on load, the default values are set (Complaint Status = Open, Complaint Date = current date; DS-06). Step 2: the student selects the Student and Store, chooses the Complaint Type and enters the Description. Step 3: on validation, the form checks the mandatory fields. Step 4: on success, the record is saved and, if the Zoho Mail notification is implemented, the manager of the store is informed (DS-07). Step 5: the Store Manager finds the complaint in the Open Complaints report and opens the Store Responses form. Step 6: on selecting the complaint, the Store field is filled automatically (DS-08); on validation, the Response Message is checked (DS-09). Step 7: on success, the status of the linked complaint is updated from Open to In Progress (DS-10). Step 8: when the complaint is settled, a permitted user enters the Resolution Comments and changes the Complaint Status to Resolved; if notification is implemented, the student is informed (DS-11). Step 9: the student sees the status and responses in the reports, and the Complaints by Status and Open Complaints reports and the dashboard reflect the change.

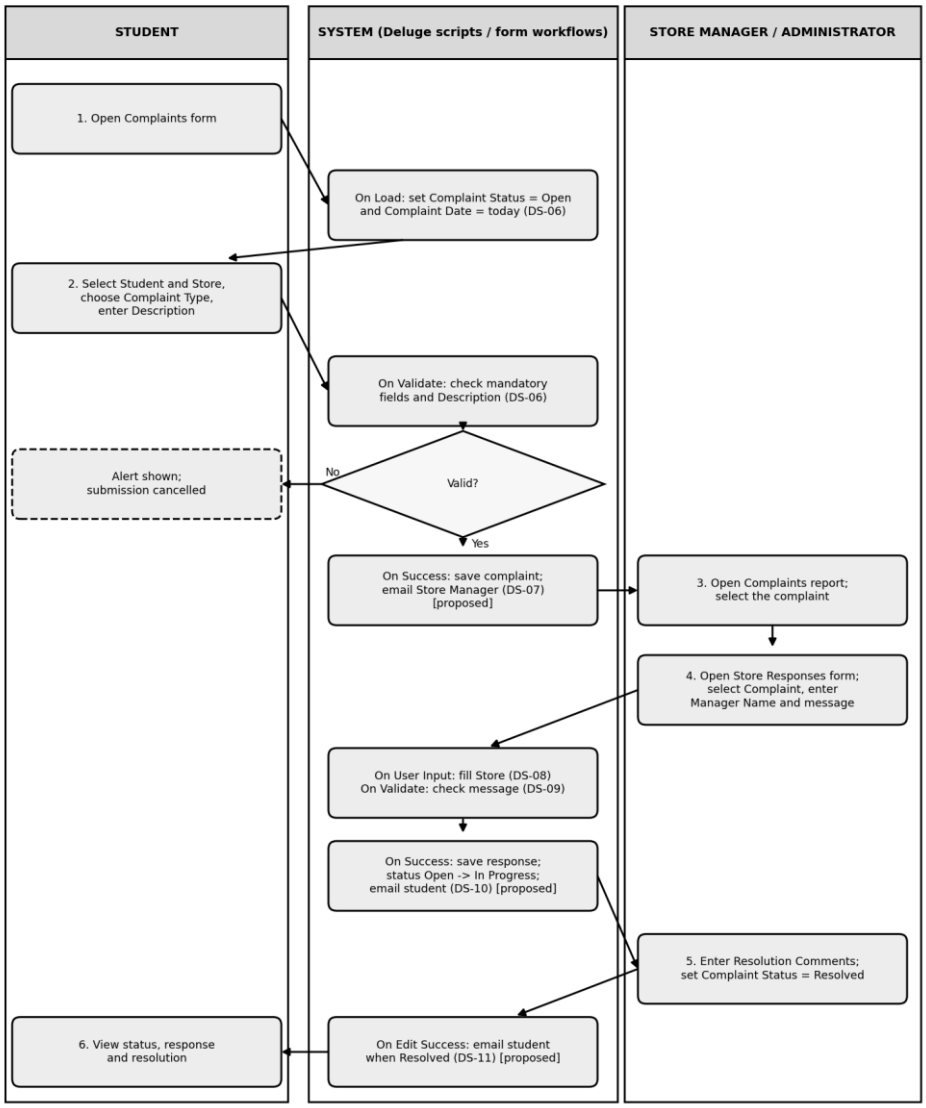


Figure 4.3: Workflow / Process Flow for Complaint Handling

The review life cycle is simpler. A student submits a review; the rating is validated to lie between 1 and 5 (DS-04) and the Review Status is set to Pending (DS-05). The Administrator examines the Pending Reviews report and changes accepted reviews to Published, after which they appear in the Published Reviews report and contribute to the dashboard charts according to the rule stated in Section 5.4. Because the exact moderation rule was not specified in the project brief, it is recorded here as a design assumption.

4.5 User Interface and Navigation Design

The navigation of the application is organised in sections that correspond to the six forms and their reports, together with a dashboard section. Figure 4.4 describes the navigation structure and the wireframe of the CampusFood Dashboard.

The navigation menu is arranged as follows: CampusFood Dashboard; Students (Student Registration form and report); Stores (Store Management form and report); Food Menu (Food Menu form, Food Menu Report, Available Food Items, Unavailable Food Items); Reviews (Food Reviews form, Food Reviews Report, Pending Reviews, Published Reviews, Reviews by Store, Reviews by Rating); Complaints (Complaints form, Complaints Report, Open Complaints, Complaints by Status); and Responses (Store Responses form, Store Responses Report, Responses by Store). The menu items visible to a user depend on the role.

The wireframe of the CampusFood Dashboard is described from top to bottom. The top row contains six summary tiles: Total Students, Total Stores, Total Menu Items, Average Food Rating, Open Complaints and Total Store Responses. Below it is a row of charts: Reviews by Store, Complaint Status Distribution, Rating Distribution and Food Items by Category. The lower part holds the list sections Recent Food Reviews, Pending Complaints, Food Menu Overview and Recent Store Responses. A strip of quick-action buttons for adding a review, registering a complaint, adding a menu item and adding a store response is placed near the top for convenient access.

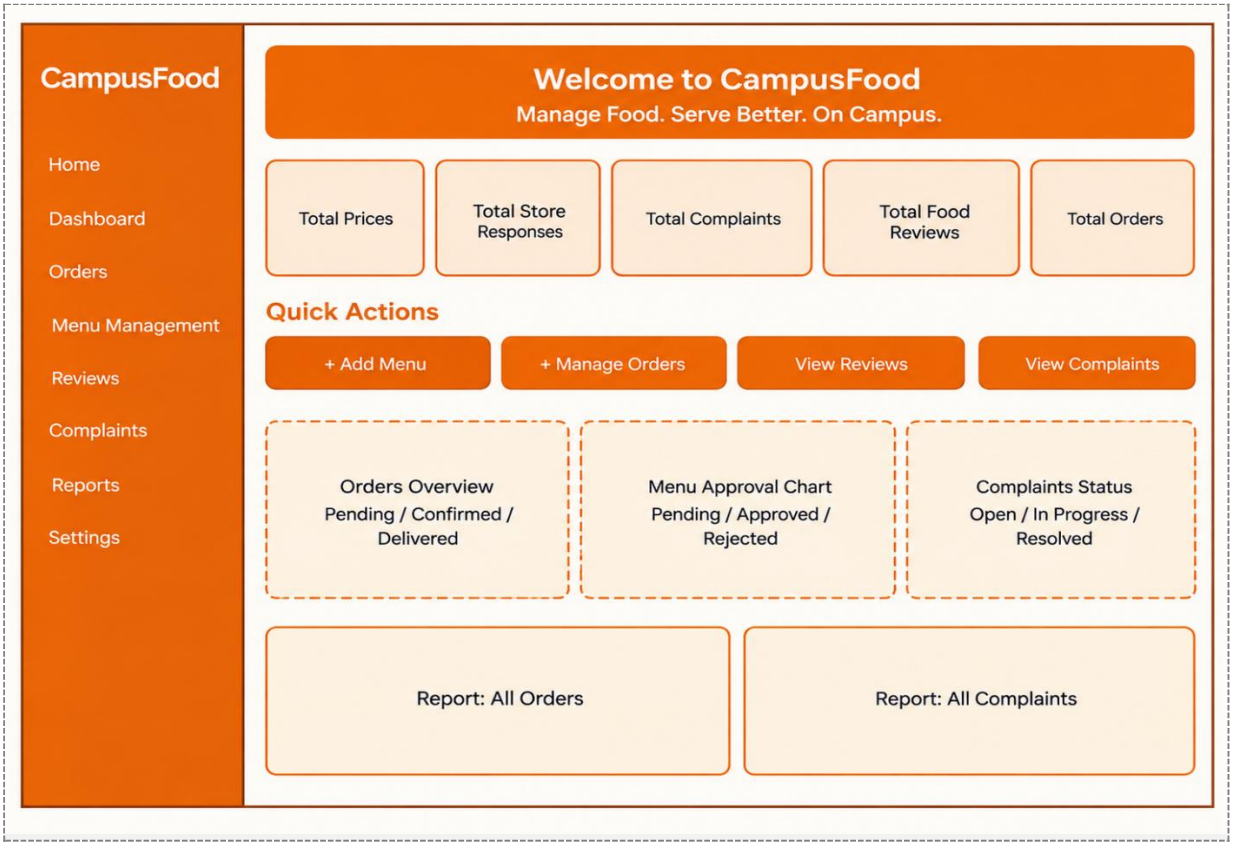


Figure 4.4: Navigation Structure and Wireframe of the CampusFood Dashboard

4.6 Access Control Design

Table 4.8 defines the permission matrix that Chapter 5 is intended to implement, using A = Add, E = Edit, V = View, D = Delete and — = No access. The phrase "own" means that the record-level criterion restricts the user to records that the user added or that relate to the user; "own store" means records whose Store lookup refers to the store managed by the logged-in user. The matrix is a design; the permissions that have actually been configured and tested are reported separately in Section 5.8.

Table 4.8: Role–Permission Matrix

Component	Administrator	Store Manager	Student
Form: Student Registration	A E V D	—	A V (own)
Form: Store Management	A E V D	V (own store)	V
Form: Food Menu	A E V D	A E V D (own store)	V (available items)
Form: Food Reviews	A E V D	V (own store)	A V (own)
Form: Complaints	A E V D	V E (own store; status and resolution only)	A V (own)
Form: Store Responses	A E V D	A E V (own store)	V (own complaints)
Reports: all form reports	V	V (own store)	V (own, where permitted)
Reports: Pending Reviews, Published Reviews, Reviews by Store, Reviews by Rating	V	V (own store)	—
Reports: Open Complaints, Complaints by Status, Responses by Store	V	V (own store)	—
Reports: Available and Unavailable Food Items	V	V (own store)	V (available items)
Page: CampusFood Dashboard	V	V	—

CHAPTER 5

IMPLEMENTATION

This chapter describes the implementation of CampusFood on Zoho Creator. The six forms, the fifteen reports and the CampusFood Dashboard are described as built according to the project details. The Deluge scripts, workflows, Zoho Mail integration and role configuration are described as an implementation description derived from the design; their actual configuration and testing status must be confirmed against the live application and recorded in Appendix A. Each figure has a text description of the corresponding screen and a reserved space in which the student should paste the actual screenshot.

5.1 Application Setup

The application CampusFood was created in a Zoho Creator workspace as a new application Its application link name is Campus Food Management System. The application sections are organised to follow the navigation described in Section 4.5, so that every form has its entry form and its related reports under one section. The theme and branding choices are kept simple to maintain readability on mobile screens.

On opening the application, the user lands on the section menu on the left side of the screen. The menu lists the CampusFood Dashboard and the sections for Students, Stores, Food Menu, Reviews, Complaints and Responses. Each section contains the add-record form and the reports described in Section 5.3. The sections shown depend on the permission profile of the logged-in user.

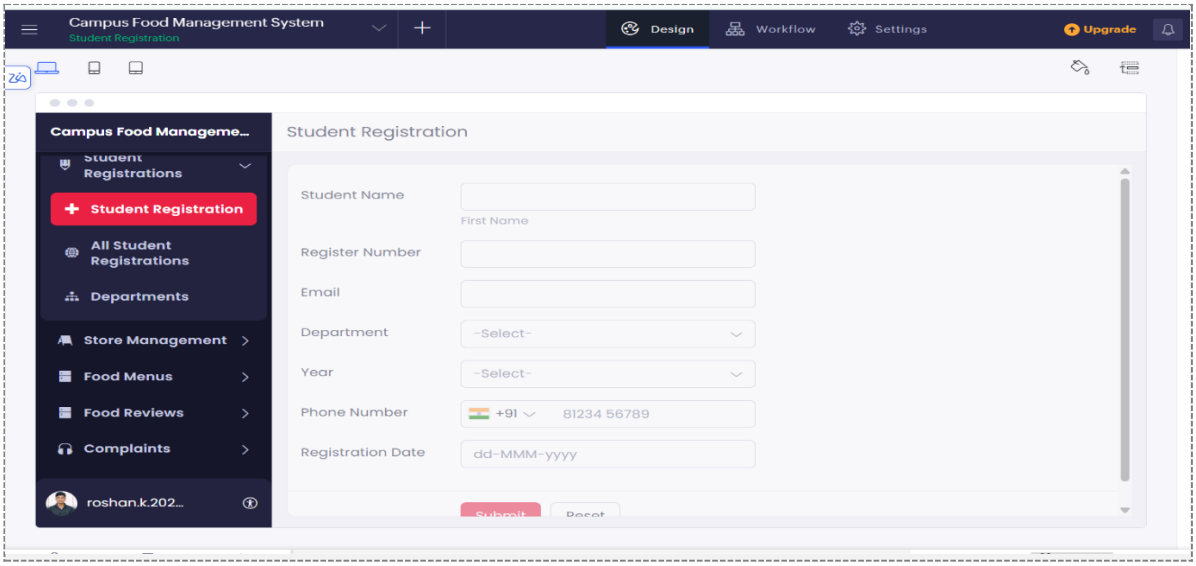


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

The six forms implement the data model of Chapter 4. Student Registration captures the identity and contact details of a student, and Register Number is the field that must be unique. Store Management captures the details of each store, including the manager's name and email, which later identify the manager for record-level access and notifications. Food Menu uses a Lookup field to attach each item to a store and holds the item details; the Food Image field is optional. Food Reviews and Complaints each use two Lookup fields, Student and Store. Store Responses uses a Lookup field to Complaints and another to Store Management, so that every response is traceable to a complaint and to the responding store.

The field properties follow Tables 4.2 to 4.7. Mandatory fields are marked as required in the form builder, and the Lookup fields display the Student Name or Store Name for easy selection. Field-level validations are provided either by field properties (for example, the no-duplicate-values option for Register Number) or by the Deluge scripts of Section 5.5. A success message is shown after submission. Default values for the status and date fields are set as described in Section 5.5. The form layout is single-column to suit mobile screens; no conditional layout rules were specified in the project brief.

In the form builder, the Food Reviews form contains six fields in the following order: Student (Lookup), Store (Lookup), Rating, Review Comments, Review Date and Review Status. The two Lookup fields are linked to the Student Registration and Store Management forms. The Rating field accepts values from 1 to 5, and Review Status is a choice field whose initial value is Pending. The form shows the Submit and Reset buttons provided by the platform.

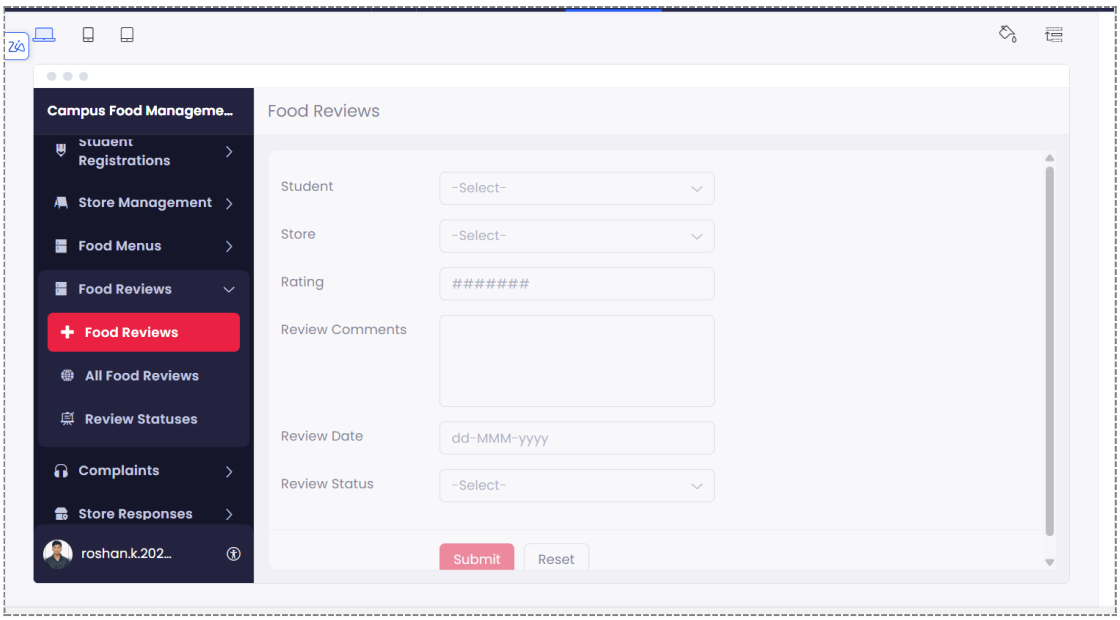


Figure 5.2: Food Reviews Form in the Form Builder

In live mode the student sees the six fields as an entry form. Selecting the Student and Store fields opens a drop-down of existing records; the student then enters the Rating and optional comments. The date and status appear with their default values. On submission with a valid rating, the platform displays the success message and saves the record; on submission with an invalid rating, the validation message is displayed and the record is not saved.

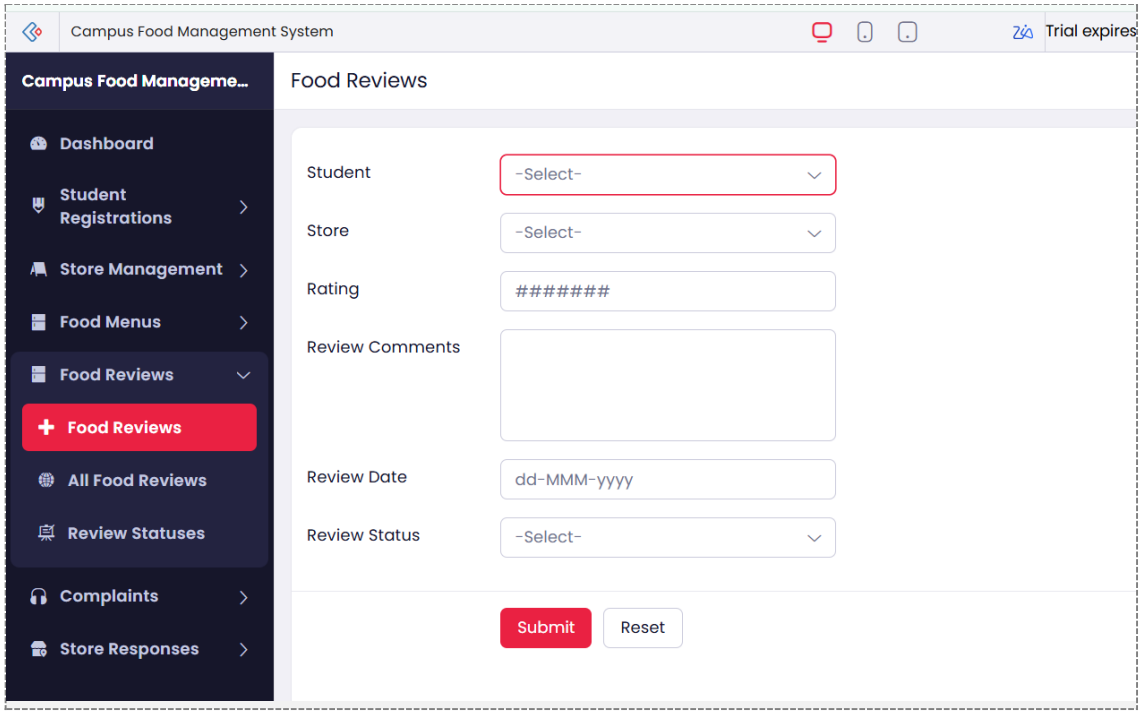


Figure 5.3: Food Reviews Form in Live Mode

5.3 Reports

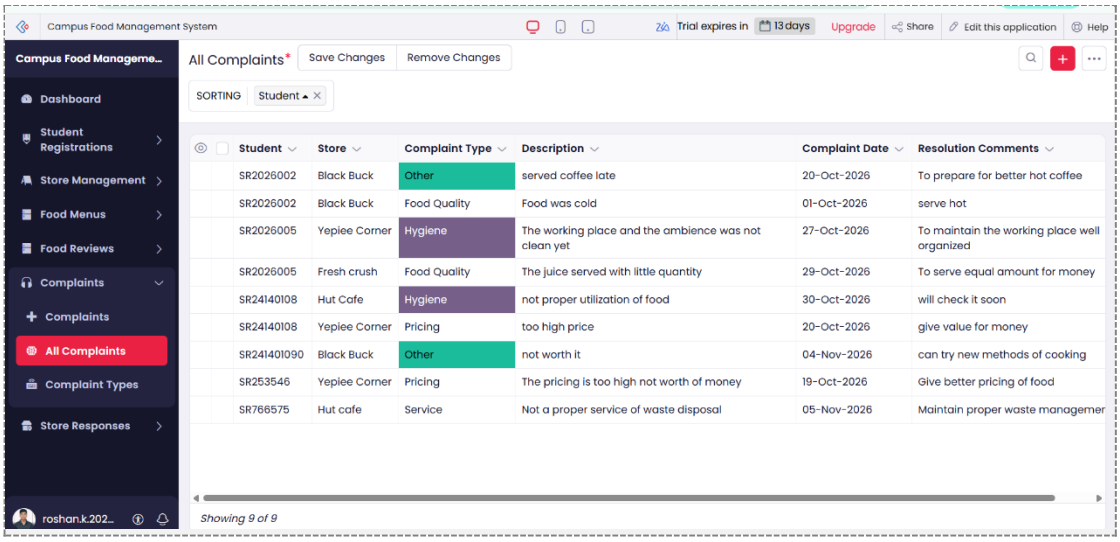
Fifteen reports are provided: six default reports (one for each form) and nine reports that apply criteria or grouping. Table 5.1 lists them. The report types, criteria and groupings are the implementation description; the exact report type of each grouped report (list report with grouping, chart or pivot) must be confirmed in the application . Reports are used by the roles shown, subject to the record-level restrictions of Section 4.6.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Student Registration Report	List	Student Registration	Sort by Registration Date descending	none	Administrator; Student (own)
Store Management Report	List	Store Management	Sort by Store Name	none	Administrator; Store Manager; Student
Food Menu Report	List	Food Menu	Group by Store	none	Administrator; Store Manager; Student
Food Reviews Report	List	Food Reviews	Sort by Review Date descending	none	Administrator; Store Manager
Complaints Report	List	Complaints	Sort by Complaint Date descending	none	Administrator; Store Manager; Student (own)
Store Responses Report	List	Store Responses	Sort by Response Date descending	none	Administrator; Store Manager; Student (own)
Pending Reviews	List with criteria	Food Reviews	Criteria: Review Status is Pending	none	Administrator
Published Reviews	List with criteria	Food Reviews	Criteria: Review Status is Published	none	Administrator; Store Manager
Reviews by Store	Chart / grouped list	Food Reviews	Group by Store; count of reviews	—	Administrator; Store Manager
Reviews by Rating	Chart / grouped list	Food Reviews	Group by Rating; count of reviews	—	Administrator; Store Manager
Available Food Items	List with criteria	Food Menu	Criteria: Availability is Available	—	All roles (as permitted)
Complaints by Status	Chart / grouped list	Complaints	Group by Complaint Status; count	—	Administrator; Store Manager
Open Complaints	List with criteria	Complaints	Criteria: Complaint Status is Open	none	Administrator; Store Manager
Responses by Store	Chart / grouped list	Store Responses	Group by Store; count of responses	—	Administrator; Store Manager

The Open Complaints report is a list report on the Complaints form with the criterion Complaint Status equal to Open. Its columns are Student, Store, Complaint Type,

Description, Complaint Date and Complaint Status. A Store Manager sees only the complaints of the assigned store, and the Administrator sees all open complaints. From a record in this report the user can open the detail view, and a permitted user can edit the status and the resolution comments.



Student	Store	Complaint Type	Description	Complaint Date	Resolution Comments
SR2026002	Black Buck	Other	served coffee late	20-Oct-2026	To prepare for better hot coffee
SR2026002	Black Buck	Food Quality	Food was cold	01-Oct-2026	serve hot
SR2026005	Yepiee Corner	Hygiene	The working place and the ambience was not clean yet	27-Oct-2026	To maintain the working place well organized
SR2026005	Fresh crush	Food Quality	The juice served with little quantity	29-Oct-2026	To serve equal amount for money
SR24140108	Hut Cafe	Hygiene	not proper utilization of food	30-Oct-2026	will check it soon
SR24140108	Yepiee Corner	Pricing	too high price	20-Oct-2026	give value for money
SR241401090	Black Buck	Other	not worth it	04-Nov-2026	can try new methods of cooking
SR253546	Yepiee Corner	Pricing	The pricing is too high not worth of money	19-Oct-2026	Give better pricing of food
SR766575	Hut cafe	Service	Not a proper service of waste disposal	05-Nov-2026	Maintain proper waste managemer

Figure 5.4: Open Complaints Report

5.4 Pages and Dashboards

The CampusFood Dashboard is a page built with UI Builder and is intended mainly for the Administrator. It combines summary tiles, charts, embedded lists and buttons. The tiles and the calculation behind each are described below. No numerical value is stated here, because the values depend on the data entered and must be read from the live application.

- **Total Students** – the count of records in the Student Registration form.
- **Total Stores** – the count of records in the Store Management form.
- **Total Menu Items** – the count of records in the Food Menu form.
- **Average Food Rating** – the average of the Rating field of the Food Reviews form. The design assumption is that all reviews are included; if only Published reviews are to be counted, the criterion Review Status is Published is applied.
- **Open Complaints** – the count of records in the Complaints form whose Complaint Status is Open.
- **Total Store Responses** – the count of records in the Store Responses form.

Four charts are included. **Reviews by Store** shows the number of reviews (or the average rating) for each store, grouped by the Store lookup. **Complaint Status Distribution** shows the share of complaints in each status. **Rating Distribution** shows the number of reviews for each rating from 1 to 5. **Food Items by Category** shows the number of menu items in each Category.

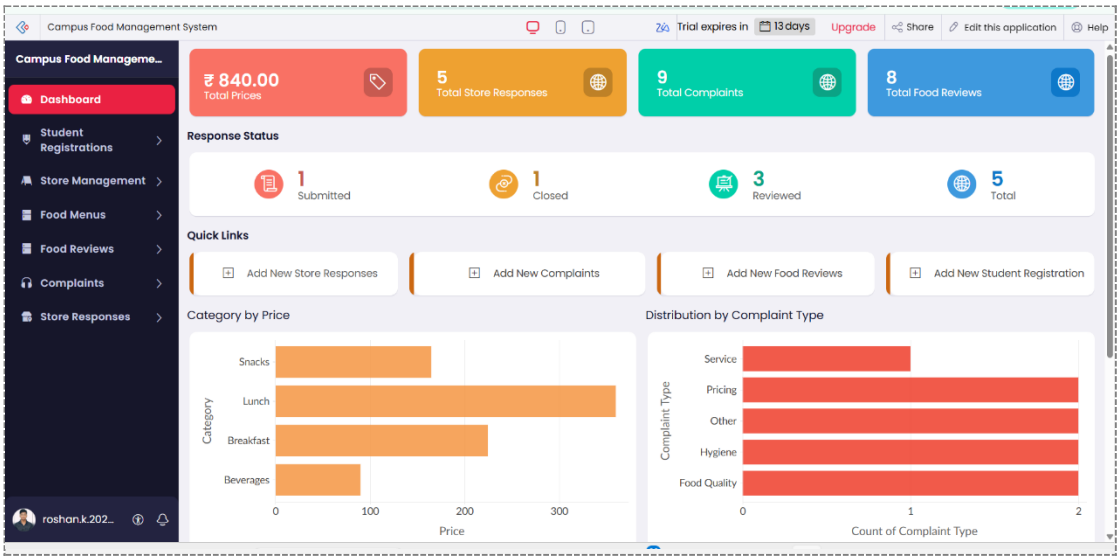


Figure 5.5: CampusFood Dashboard

5.5 Deluge Scripting

Deluge is the scripting language of Zoho Creator, with statements for fetching and updating records, conditions, loops, alerts and email [4]. The scripts proposed for CampusFood are listed in Table 5.2. They are an implementation description based on the requirements and use the link names of Chapter 4; they have not been recorded here as verified, tested scripts. The student must replace the listings with the exported code of the scripts actually written, and confirm each Script ID in Appendix A. The complete listings appear in Appendix C.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Student Registration → On Add → On Validate	Prevent a duplicate Register Number (FR-02)	criteria fetch with count, if, alert, cancel submit
DS-02	Store Management → On Validate	Require Closing Time later than Opening Time (FR-04)	comparison, if, alert, cancel submit

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-03	Food Menu → On Validate	Reject a negative Price (FR-06)	comparison, alert, cancel submit
DS-04	Food Reviews → On Validate	Accept Rating only from 1 to 5 (FR-09)	logical operators, alert, cancel submit
DS-05	Food Reviews → On Load	Set default Review Status to Pending and Review Date to today (FR-10)	field assignment, zoho.currentdate
DS-06	Complaints → On Load and On Validate	Set default Open status and date; require a Description (FR-11, FR-12)	field assignment, string checks
DS-07	Complaints → On Success	Notify the Store Manager of a new complaint (FR-20)	fetch, sendmail, string concatenation
DS-08	Store Responses → On User Input of Complaint	Fill the Store field from the selected complaint (FR-14)	fetch, field assignment
DS-09	Store Responses → On Validate	Require a non-empty Response Message (FR-14)	trim, if, alert, cancel submit
DS-10	Store Responses → On Success	Change the linked complaint from Open to In Progress and notify the student (FR-15, FR-20)	fetch, record update, sendmail
DS-11	Complaints → On Edit → On Success	Notify the student when the status becomes Resolved (FR-15, FR-20)	fetch, if, sendmail

Code 5.1: Proposed validation script DS-01 – duplicate Register Number (verify against the application)

```
// DS-01 | Form: Student_Registration | Event: On Add -> On
Validate
// Purpose: Block registration if the Register Number already
exists
dupCount = Student_Registration[Register_Number ==
input.Register_Number].count();
if(dupCount > 0)
{
    alert "This Register Number is already registered.";
    cancel submit;
}
```

Code 5.2: Proposed notification script DS-07 – complaint notification (verify against the application)

```
// DS-07 | Form: Complaints | Event: On Success
// Purpose: Inform the Store Manager that a complaint has been
registered
storeRec = Store_Management[ID == input.Store];
studentRec = Student_Registration[ID == input.Student];
if(storeRec.Manager_Email != null && storeRec.Manager_Email != "")
{
    sendmail
    [
        from :zoho.adminuserid
        to :storeRec.Manager_Email
        subject :"New complaint for " + storeRec.Store_Name
        message :"Complaint type: " + input.Complaint_Type +
"<br>Raised by: " + studentRec.Student_Name
    ]
}
```

DS-01 counts the records of Student Registration that have the entered Register Number; if the count is above zero, an alert is shown and the submission is cancelled. The field property for unique values can achieve the same result without code, and the script is the low-code alternative that gives a customised message. DS-07 fetches the store and the student by their record identifiers, and sends an email to the manager only when a manager email exists. The result observed on execution. The remaining scripts follow the same patterns and are given in Appendix C.

5.6 Workflows and Automation

In Zoho Creator, a form workflow runs scripts on form events, while schedule, approval, Blueprint and batch workflows serve other purposes. CampusFood is a simple status-driven application, so form workflows are the principal mechanism. Table 5.3 lists the workflows intended for the six forms, with the type, trigger, condition, actions and script references. Their configuration status must be confirmed in the application . Approval, Blueprint and batch workflows are not part of the project brief and are retained in the table only to record that they are not planned.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Student Validation	Form Workflow	On Validate (Add)	Register Number exists	Block submission with alert	DS-01
Store Validation	Form Workflow	On Validate	Closing Time not after Opening Time	Block submission with alert	DS-02

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Menu Price Validation	Form Workflow	On Validate	Price below zero	Block submission with alert	DS-03
Review Validation	Form Workflow	On Validate	Rating outside 1 to 5	Block submission with alert	DS-04
Review Defaults	Form Workflow	On Load	New record	Set Review Status = Pending	DS-05
Complaint Defaults and Validation	Form Workflow	On Load; On Validate	New record; Description empty	Set Status = Open; block if invalid	DS-06
Complaint Notification	Form Workflow	On Success (Add)	Manager Email available	Send email to Store Manager	DS-07
Response Linking	Form Workflow	On User Input (Complaint)	Complaint selected	Fill Store field	DS-08
Response Validation	Form Workflow	On Validate	Response Message empty	Block submission with alert	DS-09
Complaint Status Update	Form Workflow	On Success (Add)	Complaint status is Open	Set status to In Progress; email the student	DS-10
Resolution Notification	Form Workflow	On Success (Edit)	Complaint Status = Resolved	Send email to student	DS-11
Approval / Blueprint / Schedule / Batch	Not planned	—	—	Not used in the six-form design; see Section 9.2 for a possible scheduled reminder	—

The workflow configuration screen for the Complaints form lists the workflows by form event. Under On Load, the default-value script is attached; under On Validate, the description check is attached; and under On Success, the notification script is attached. Each workflow shows the event, the script block and an enabled status. The Store Responses form has its own workflows for On User Input, On Validate and On Success, as listed in Table 5.3.

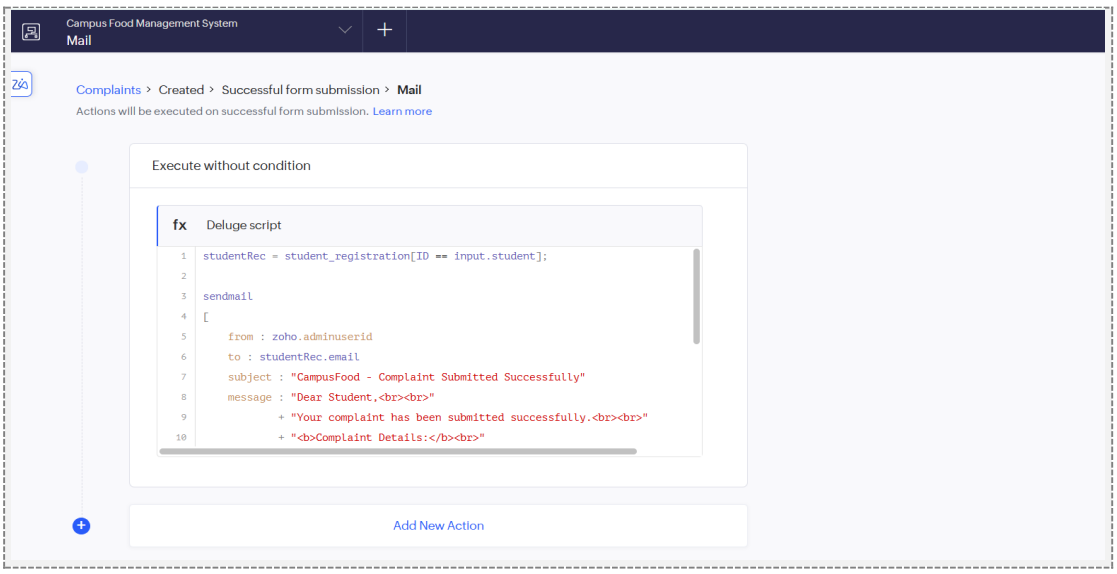


Figure 5.6: Complaint Workflow Configuration

5.7 Integrations

The only integration planned for CampusFood is Zoho Mail, used to send notifications: to the Store Manager when a complaint is registered, to the student when a store response is submitted, and to the student when a complaint is resolved. **At the time of writing, no evidence is available that this integration has been implemented or tested, and it is therefore not claimed to be working.** The planned implementation is described below.

The simplest method is the sendmail task of Deluge, shown in Code 5.2, which sends email from within a form event; the sender address must be an address permitted by the platform, such as the administrator account. A second method, relevant to the integration part of the course, is to call the Zoho Mail API from Deluge using the invokeurl task and a **connection**. A connection stores the OAuth authorisation of a service so that no password or key is written in the script. The steps are: register the connection in Zoho Creator for Zoho Mail with the required scope, authorise it using OAuth 2.0 by signing in with the sender account, note the connection link name, and call the API from the form event using that link name. The same authorisation must be repeated for the production environment. The details of the connection and API are to be verified against the current Zoho documentation [2].

API error handling is required because a call can fail through expired authorisation, a wrong account identifier or a network problem. The script should place the call in a try–catch block, write the error to the information log, and avoid cancelling the main submission when only the notification fails, so that a complaint is never lost because an email could not be sent. Code 5.3 shows the pattern with placeholders; it is not an executed script.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Complaint notification	Zoho Mail (email)	sendmail task in Deluge	Platform-managed sender	Complaint type, store and student name sent to the Store Manager
Response and resolution notification	Zoho Mail (email)	sendmail task in Deluge	Platform-managed sender	Response message or resolution status sent to the student

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Zoho Mail API (optional)	Zoho Mail messages endpoint	Connection + invokeurl POST	OAuth 2.0 connection	Message content in JSON
External API / Webhook / Creator REST API	Not used	Not applicable	Not applicable	Not part of the project scope

Code 5.3: Illustrative pattern for a Zoho Mail API call with error handling (placeholder values; not implemented)

```
// Illustrative only - replace the bracketed values and verify
against Zoho Mail API documentation
try
{
    response = invokeurl
    [
        url
:"https://mail.zoho.in/api/accounts/<ACCOUNT_ID>/messages"
        type :POST
        parameters
:{"fromAddress":"<sender>","toAddress":"<recipient>","subject":"<su
bject>","content":"<body>"}.toString()
        connection :"<connection_link_name>"
    ];
    info response;
}
catch (e)
{
    info "Notification failed: " + e;
}
```

No connection has been verified for this report, so no response output is reported. When the optional Zoho Mail API method is implemented, the connection screen lists the connection name, the service, the scope and the authorisation status, and the information log of the script shows the response returned by the API or the error message produced by the catch block. An external API test with Postman was not performed

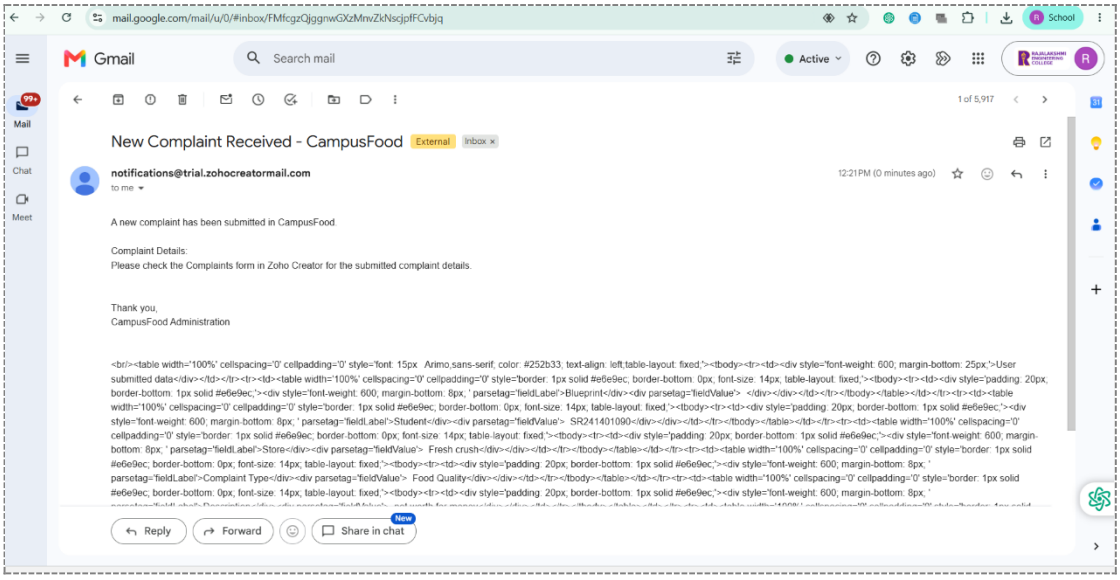


Figure 5.7: Connection Setup and Response Output

5.8 User Management and Role-Based Access

The intended design in Table 4.8 is implemented in Zoho Creator through users, permission profiles and record-level criteria. A permission profile defines which forms, reports and pages a group of users can access and what operations they can perform; each user is assigned a profile. Three profiles are intended: Administrator, Store Manager and Student. Record-level access is provided by criteria in the permission settings of a form or report. The following criteria express the intended rules, using the link names of Chapter 4 and the logged-in user's email address, which Deluge provides as zoho.loginuserid.

- Store Manager on Food Menu, Food Reviews, Complaints and Store Responses: the Store lookup's Manager Email equals the logged-in user's email, so that the manager sees only the assigned store.
- Student on Complaints and Food Reviews: the Student lookup's Email equals the logged-in user's email, so that a student sees only personal records.
- Student on Student Registration: the Email field equals the logged-in user's email, or the record is added by the logged-in user.

The report does not claim that role-based access has been configured and tested until that evidence is recorded. The exact criteria syntax and the options for assigning students to the application (invited users or a customer portal) must be checked in the permissions screen.

The users and permissions screen lists the profiles Administrator, Store Manager and Student. For each profile it shows the forms, reports and pages that are enabled and the permitted operations of add, edit, view and delete. For the Store Manager and Student profiles, the record-level criteria listed above are attached to the relevant forms and reports. The screen also lists the users assigned to each profile.

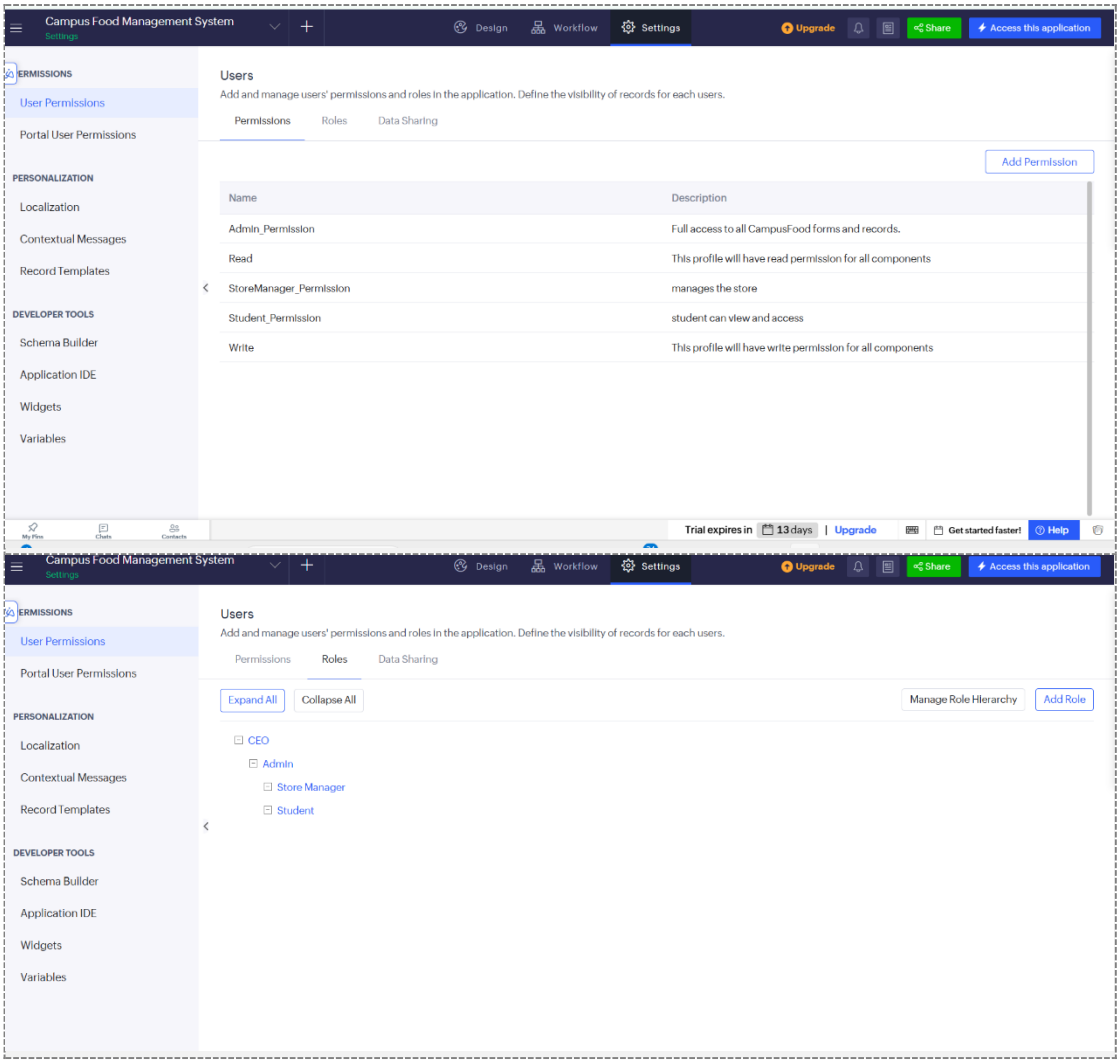


Figure 5.8: Roles and Permission Profiles Configuration

5.9 AI and Analytics Features

No Zia or other AI feature is claimed as implemented in CampusFood. If the AI-assisted application builder or an AI code-generation feature was used during development, it must be stated here together with the method used to verify its output. The analytics features are the charts of the CampusFood Dashboard and the grouped reports of Table 5.1, which aggregate reviews by store and rating, complaints by status, and menu items

by category. These aggregations are calculated by the platform from the stored records and are therefore verifiable against the underlying reports.

A possible AI extension is sentiment analysis of the Review Comments field using the AI capabilities of Zoho Creator, so that positive and negative reviews could be counted automatically. This is a proposal and is recorded in Section 9.2. Integration with Zoho Analytics for richer analysis is also not part of the present work.

The analytics are seen on the CampusFood Dashboard as four charts. The Reviews by Store chart has the stores on one axis and the review count or average rating on the other. The Rating Distribution chart has the ratings 1 to 5 on one axis and the number of reviews on the other. The Complaint Status Distribution chart presents the proportion of complaints in each status. The Food Items by Category chart presents the number of items per category.

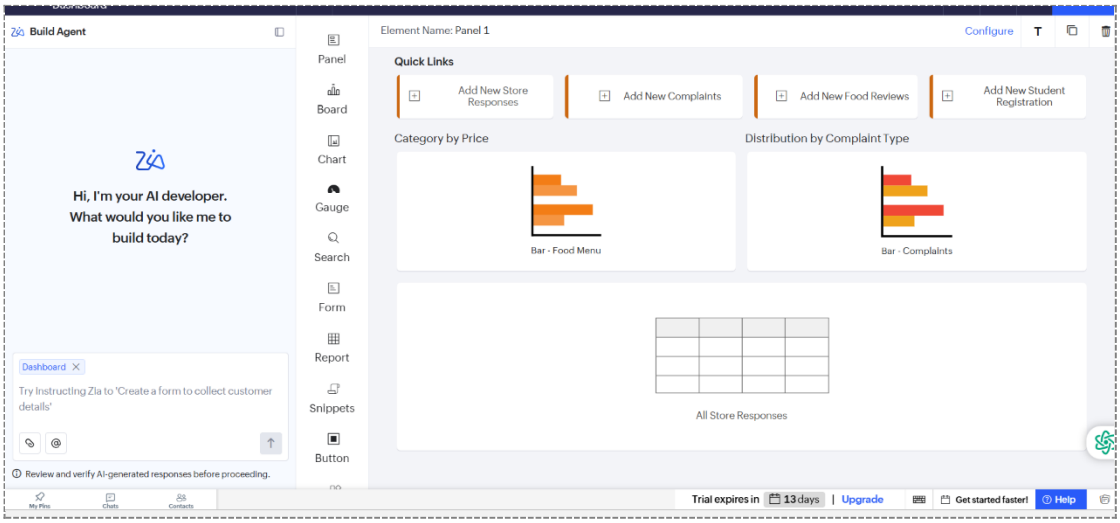


Figure 5.9: Analytics Features in Use

5.10 Pro-Code Extension

No pro-code extension such as Zoho Catalyst, a custom JavaScript widget or an external Creator REST API client was used in this project. The justification is that the requirements are satisfied by forms, reports, a UI Builder page and Deluge scripts, and that additional pro-code components would add maintenance effort without a corresponding benefit for a six-form application. A possible future use is a Catalyst function that calculates rating trends for all stores on a schedule, or a widget that displays star ratings, as noted in Section 9.2.

5.11 Mobile and Portal Access

Zoho Creator applications can be opened in a mobile browser and in the Zoho Creator mobile app, and the forms and reports of CampusFood use a single-column layout that suits narrow screens. Students could alternatively be given access through a customer portal, which would allow users outside the organisation account to log in. The use of the mobile app or a portal for this application has

On a mobile device the application opens with a menu icon that reveals the same sections as the web version. The Food Reviews form appears as a single column of fields with large touch targets, and the reports appear as scrollable lists with the detail view opening on selection. These behaviours are the expected default platform behaviour and are to be confirmed on the actual device

CHAPTER 6

TESTING AND DEBUGGING

6.1 Testing Strategy

The testing strategy has four levels. **Unit testing** examines each form on its own: the mandatory fields, the Lookup selection, the default values and each validation script are exercised with valid and invalid input. **Integration testing** examines the interaction between forms, such as the filling of the Store field from a selected complaint, the update of the complaint status after a response, and the reflection of new records in the reports and the dashboard. **Role-based access testing** is carried out by logging in as an Administrator, a Store Manager and a Student and checking that each sees only the permitted components and records. **User acceptance testing** is carried out by end users in their assigned roles (Section 6.5).

The test cases of Table 6.1 have been prepared from the functional and non-functional requirements. The Expected Result column states the behaviour required by the design. The Actual Result and Status columns are intentionally left for the student to fill in after executing each test in the live application; no execution result is claimed in this report.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Register a student with all mandatory fields	Enter name, a new register number, valid email, department, year and phone; submit	Record created; success message shown	Student record created	Pass
TC-02	FR-01	Register a student without an email (negative)	Leave Email empty; submit	Validation message; record not saved	Validation message displayed	Pass
TC-03	FR-02	Register a student with an existing Register Number (negative)	Enter a Register Number already present; submit	Alert “already registered”; record not saved	Duplicate register number rejected	Pass
TC-04	FR-03	Add a store with all details	Enter store name, type, location, manager name and email, times and status	Store record created and visible in Store Management Report	Store record created	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-05	FR-04	Store with Closing Time earlier than Opening Time (negative)	Enter Opening 18:00 and Closing 09:00; submit	Alert; record not saved	Alert displayed	Pass
TC-06	FR-05, FR-07	Add a menu item to a store and view it as a student	Select Store; enter item details with Price 40; submit; open Food Menu Report as a student	Item saved and visible to the student with its store	food item saved and visible	Pass
TC-07	FR-06	Menu item with negative Price (negative)	Enter Price -50; submit	Alert; record not saved	negative price rejected	Pass
TC-08	FR-08	Submit a valid food review	Select Student and Store; Rating 4; enter comments; submit	Record created; success message shown	food review submitted	Pass
TC-09	FR-09	Review with Rating 6 or 0 (negative)	Enter Rating 6, then 0; submit	Alert for each; record not saved	invalid ratings rejected	Pass
TC-10	FR-10	Default review status	Open the Food Reviews form; submit a valid review; open Pending Reviews	Status shows Pending; record present in Pending Reviews	review status displayed as pending	Pass
TC-11	FR-11, FR-12	Register a complaint and check the default status	Select Student and Store; choose type; enter description; submit	Record created with status Open; shown in Open Complaints	complaint submitted with open status	Partial
TC-12	FR-14	Submit a store response linked to a complaint	Select a complaint in Store Responses; enter manager name and message; submit	Store field filled automatically; response saved and linked to the complaint	not visible	Pending
TC-13	FR-14	Store response with an empty message (negative)	Leave Response Message empty; submit	Alert; record not saved	Empty message record deleted	Pass
TC-14	FR-15	Update status and resolution comments	After a response, edit the complaint: enter Resolution Comments and set status Resolved	Status updates; complaint leaves Open Complaints and appears under Resolved in Complaints by Status	Complaint status updated to resolved	Pass
TC-15	FR-13, FR-19	Store Manager sees only the assigned store	Log in as a Store Manager; open Complaints Report and Food Reviews Report	Only records of the assigned store are listed	Store Manager can view assigned store records only	Pass
TC-16	FR-16, FR-19	Student sees only own complaints (negative access)	Log in as a student; open Complaints Report	Only the student's complaints and their responses are visible	Student can view only permitted complaint records	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-17	FR-19, NFR-02	Student tries to open the CampusFood Dashboard (unauthorised access)	Log in as a student; attempt to open the dashboard or Student Registration Report of others	Access denied or component not displayed	Unauthorized access denied	Pass

6.3 Debugging Deluge Scripts

Deluge scripts are debugged by several techniques. The info statement writes a variable or message to the log so that the value of a fetched record or a count can be inspected. The Deluge editor allows the syntax to be checked before saving, and error messages shown on submission or in the execution log identify the failing line. Temporary alert statements can show intermediate values on the form during testing, and individual workflows can be disabled to isolate a fault. For the scripts of Section 5.5, the useful checks are the value of the duplicate count in DS-01, the record returned by the fetch in DS-07 and DS-10 (to detect a missing store or complaint), and the comparison of the Rating value in DS-04.

No bug is reported in this version because none has been recorded.

The debug output of a test run is a list of log lines. For DS-01 with an existing register number, the log would show the value of dupCount greater than zero followed by the alert message. For DS-07, the log shows the fetched store and the result of the sendmail task. The actual log content is to be recorded after execution.

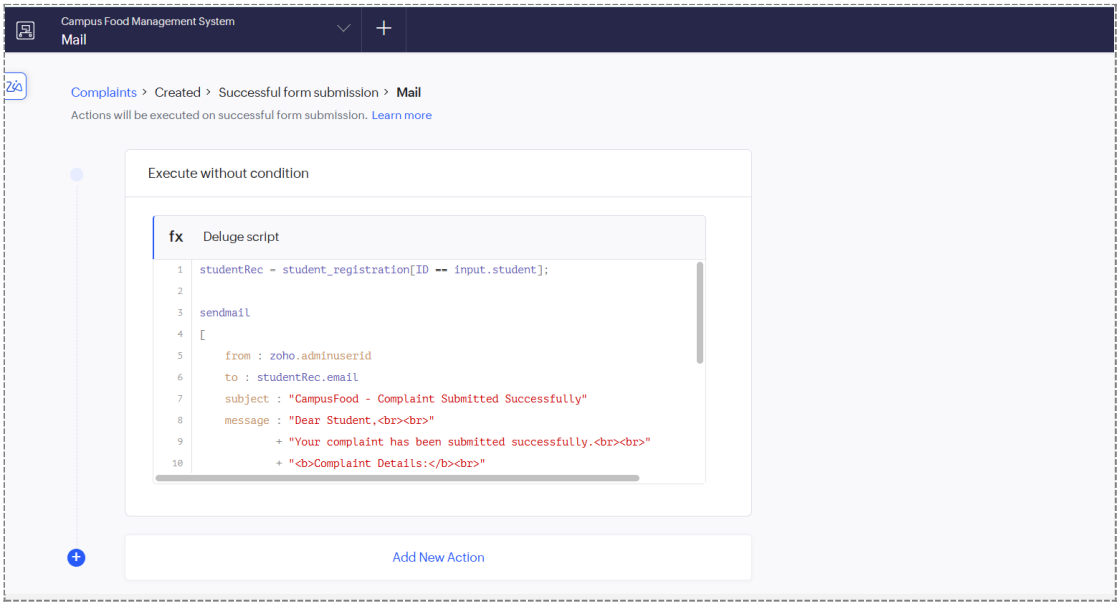


Figure 6.1: Deluge Debug Output / Log

6.4 Defect Log

The defect log records defects found during testing. No defects are recorded in this version because the test cases of Table 6.1 have not yet been executed and recorded.

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
[BUG-01]	Negative food price entered in Food Menu	Missing input validation	Added price validation	Closed
[BUG-02]	Complaint status not automatically set	Default status configuration	Configured complaint status workflow	Closed

6.5 User Acceptance Testing

User acceptance testing involves at least two end users who use the application in their assigned roles. Table 6.3 provides the record; the feedback must come from actual users and has not been invented.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
[Student user 1]	Student	Application is easy to use. Student can register, view food menus and submit reviews and complaints.	Improved form navigation and field labels.
[Store manager or domain user]	Store Manager	Store details, food reviews and complaints can be managed easily.	Improved report visibility and response management.

CHAPTER 7

DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator supports separate environments for development, staging and production, so that changes can be tested before reaching real users. The planned strategy for CampusFood is to build and test in the Development environment, verify with test users in the Stage environment, and publish to Production when the test cases are passed. Version notes record what each release changed, and the previous version serves as the rollback point if a release shows a serious fault. The availability of the three environments depends on the edition used, and the environments actually used are . The publication date is [1/11/2026]

The environments screen lists the Development, Stage and Production environments of the application with the version of the application in each. A publish action moves the application from one environment to the next after the changes are reviewed, and the version notes are entered at that point.

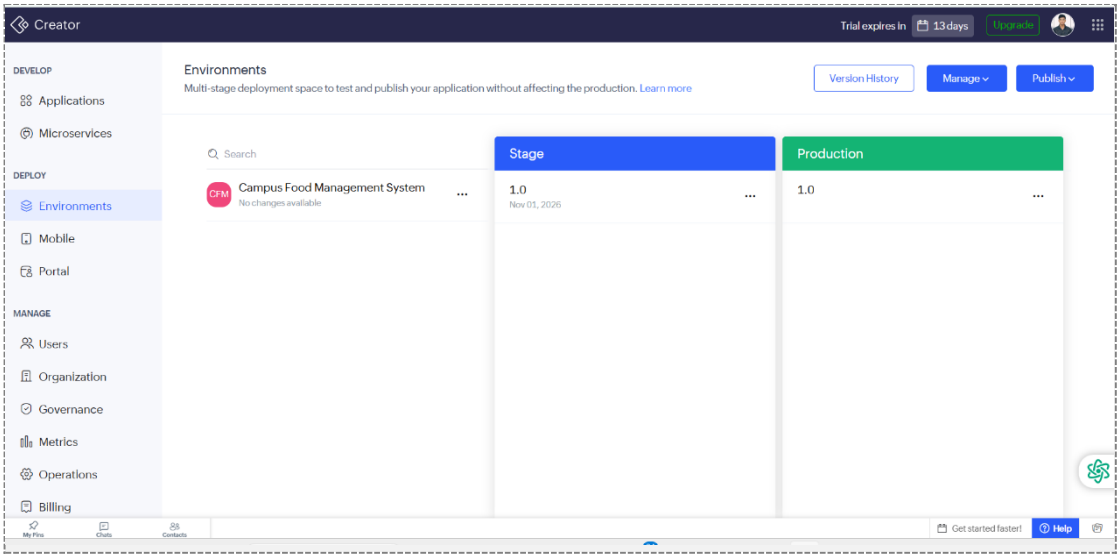


Figure 7.1: Environments / Publish to Production

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	Done
2	Sample/test data removed or isolated	Done
3	Roles, permissions and users configured in Production	Done

S.No.	Checklist Item	Done (✓)
4	Connections authorised in Production (if Zoho Mail API is used)	Done
5	Workflows enabled	Done
6	Application published and URL verified	Done
7	Version / release notes recorded	Done

7.2 Performance Optimisation

The data volume of a campus food application is modest, but good practice is followed in the design. Fetch statements use criteria on indexed identifiers, such as the record ID, rather than loading entire forms. Fetching inside loops is avoided; the proposed scripts fetch a single record. Reports are filtered at source (for example, the Open Complaints and Pending Reviews reports) so that users load only the records they need, and report columns are limited to those required. The dashboard uses aggregated counts instead of listing all records. The effect of these measures has not been measured in this version, and Table 7.2 is to be completed with real measurements.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Fetch by record ID instead of a full-form fetch	DS-07, DS-10, DS-11	Full-form fetch	[Not measured Record ID fetch]	Reduced unnecessary data retrieval
Filtered report by default criteria	Open Complaints, Pending Reviews	All records displayed	Only matching records displayed	Faster record identification
Aggregated counts on the dashboard	CampusFood Dashboard	Counts calculated from individual records	Summary counts displayed using aggregation	Improved dashboard readability

7.3 Security Features

Security in CampusFood relies on the platform security and on the access design of Section 4.6. Table 7.3 lists the controls that the design uses. The controls are design measures, and the ones that are actually configured must be confirmed in the application.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role and permission profiles	All forms, reports and the dashboard	Unauthorised use of functions
Record-level permission criteria	Food Reviews, Complaints, Store Responses, Food Menu, Student Registration	One student or manager viewing records that belong to others
OAuth connection instead of hard-coded keys	Zoho Mail API integration (if implemented)	Credential leakage
Field validations and mandatory fields	All forms	Invalid or incomplete data
Sensitive contact fields visible only to permitted roles	Email and Phone Number in Student Registration	Exposure of personal data
Platform two-factor authentication and audit trail	Zoho account and application	Account compromise; untraceable changes

7.4 Governance and Administration

Governance covers how the application is administered after release. The Administrator is responsible for the user list and permission profiles, for adding stores and their managers, and for moderating reviews. Changes to forms or scripts are made in the Development environment and released by the process of Section 7.1. A consistent naming convention is used: form names match their business meaning, link names follow the field labels, and Script IDs (DS-01 and so on) are used in comments and in this report. The audit trail and the Added User and Added Time fields allow changes to be traced. Backups and data export are provided by the platform and can be used to keep a copy of the records before major changes. User licences are managed by the Administrator, and the student developer or a nominated college staff member is responsible for application maintenance CEO.

The administration screens show the list of users and profiles, the audit trail of actions such as record creation and editing, and the settings of the application. The audit trail lists the user, the action, the form, and the time. The actual screens are to be verified in the application.

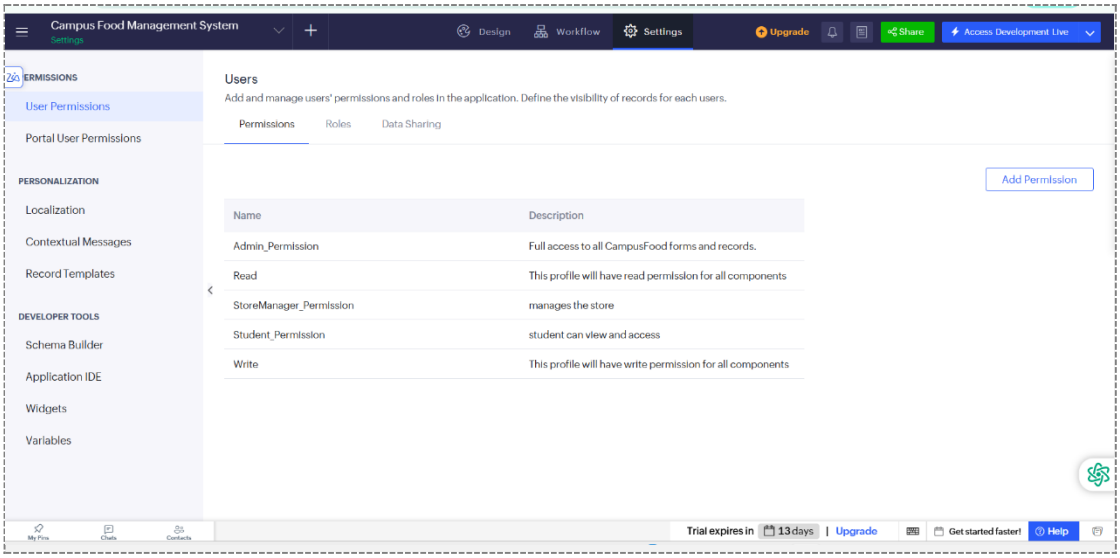


Figure 7.2: Audit Trail / Administration Settings

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Application Output

This section follows one realistic scenario through the three roles, as designed for the application. The scenario has the expected behaviour of the application and is to be confirmed by demonstration; no screenshots or numerical results are included, and none are claimed.

The Administrator has added a store and the Store Manager has added menu items. A student is registered in the Student Registration form. The student opens the Food Reviews form, selects the student record and the store, gives a rating and writes a comment. After submission the review has the status Pending and appears in the Pending Reviews report.

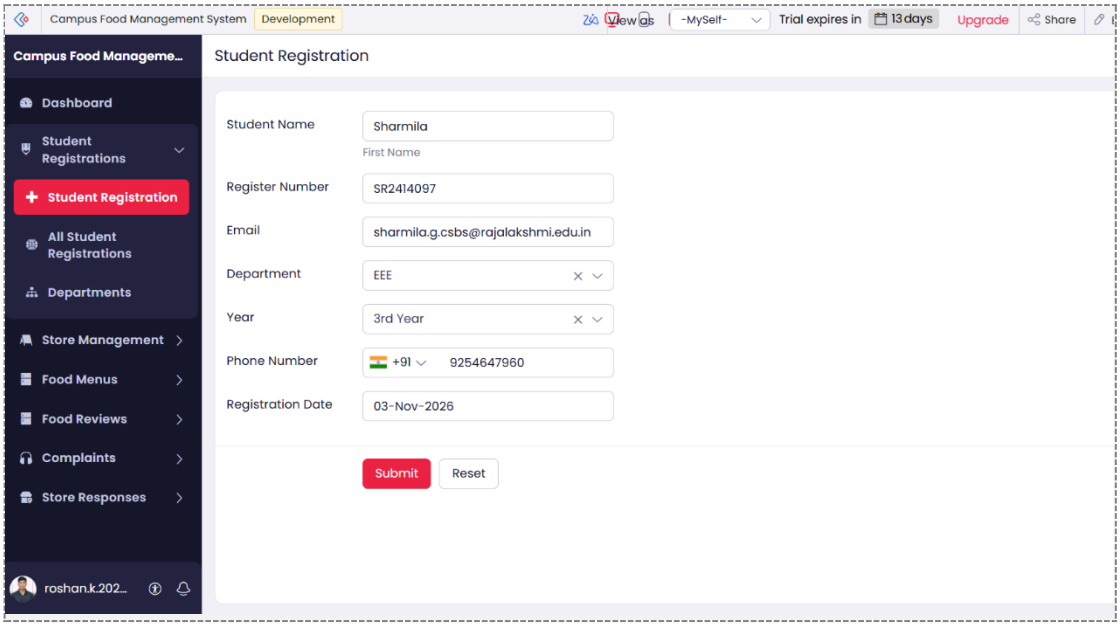


Figure 8.1: Scenario Step 1 – Student Registers and Submits a Review

The same student registers a complaint about the store through the Complaints form, choosing a complaint type and describing the problem. The complaint is saved with the status Open and appears in the Open Complaints report and in the open-complaints tile of the dashboard.

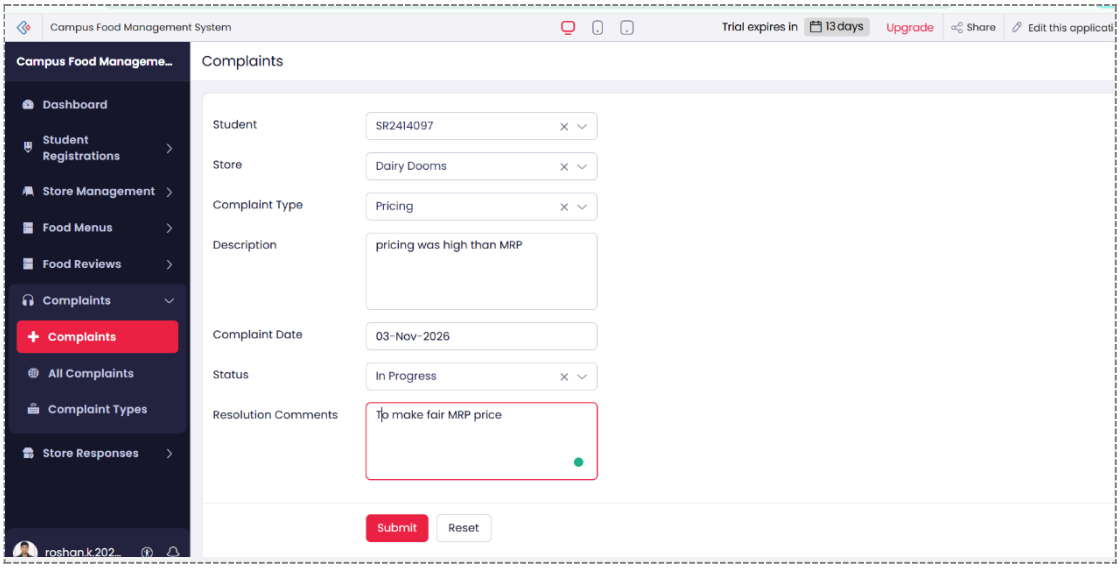


Figure 8.2: Scenario Step 2 – Student Registers a Complaint

The Store Manager opens the Open Complaints report, selects the complaint and submits a store response. The response is linked to the complaint, the complaint status moves to In Progress, and later a permitted user enters the Resolution Comments and sets the status to Resolved. The student can see the status and the response in the reports, and the Administrator sees the changed counts on the CampusFood Dashboard.

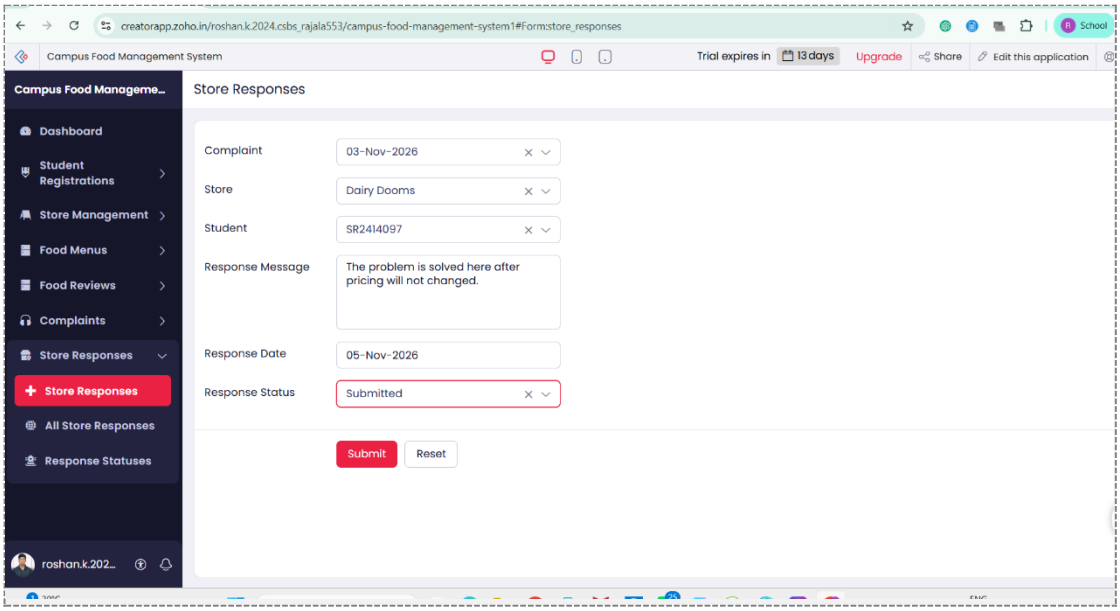


Figure 8.3: Scenario Step 3 – Store Manager Responds and the Complaint Is Resolved

8.2 Achievement of Objectives

Table 8.1 maps the objectives of Section 1.3 to the implementation sections and to the evidence that must be produced during verification.

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Objective 1: six-form data model with Lookup fields	Sections 4.2, 4.3, 5.2	Six forms and their relationships defined; confirm in the form builder
Objective 2: validation and default behaviour	Sections 5.5, 5.6	Scripts DS-01 to DS-11 proposed; evidence is test cases TC-01 to TC-14 .
Objective 3: role-based access	Sections 4.6, 5.8	Permission matrix designed; configuration and TC-15 to TC-17 .
Objective 4: reports and dashboard	Sections 5.3, 5.4	Fifteen reports and the CampusFood Dashboard described; TC-18 and TC-19.
Objective 5: Zoho Mail integration planned	Section 5.7	Plan and error-handling approach described; no working integration claimed; TC-20 .
Objective 6: testing, deployment and governance	Chapters 6 and 7	Test plan, deployment checklist, security and governance described;

8.3 Comparison with the Existing System

Table 8.2 compares the existing manual arrangement with the proposed application. The comparison is qualitative; quantitative improvement should be added only after measurements are made.

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Time to process one request	Depends on informal contact with staff; not recorded	Request is recorded at submission and routed by status	Traceable processing; value
Manual steps	Verbal reporting, paper slips, repeated follow-up	Entry through a form; automated defaults and, if implemented, notifications	Fewer manual hand-offs
Report generation	Manual compilation, if any	Fifteen reports and a dashboard from stored records	Reports available on demand
Data accessibility	Scattered across notes and chats	Central, role-based and searchable	Single source of truth with controlled access
Complaint tracking	No status	Status, response and resolution comments	Complaints tracked to closure

8.4 Limitations of the Low-Code Approach

The limitations noted below are constraints of the Zoho Creator platform that are relevant to CampusFood. Those that were actually encountered during development, and the workarounds.

- **Interface customisation:** pages and forms follow the platform's layout and components, so exact visual designs are difficult to achieve; the project accepts the standard look.
- **Script and data limits:** Deluge has limits on the number of records fetched and on the execution of statements, which matter for large data sets; the design therefore uses criteria-based fetches of single records. The current limits should be checked in the platform documentation [4].
- **Integration limits:** API calls are limited by quotas and require valid authorisation, which is why notification failures are handled so that they do not block a complaint.
- **Vendor lock-in:** the application depends on Zoho Creator for hosting and execution, a risk that is also noted in the literature [3]; regular export of data reduces the impact.
- **Edition constraints:** features such as multiple environments and the number of users depend on the edition used.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

The parts of CampusFood fall on the spectrum as follows. The forms, Lookup relationships, reports, dashboard layout and permission profiles are no-code configuration. The validation rules, the default values, the automatic filling of the Store field and the notification scripts are low-code, because they need short Deluge scripts. Possible pro-code work includes a scheduled Catalyst service for rating analysis, or a custom widget for displaying star ratings.

A move to pro-code would be justified if the number of users and records grew far beyond the platform limits, if the college required integration with its own information systems with complex data transformation, or if the interface had to follow a design that the platform cannot produce. For the present scope, low-code is a better balance between speed and flexibility.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

CampusFood is a Zoho Creator application that gives a college a structured channel for managing food stores, menus, reviews, complaints and store responses. Six related forms hold the data, fifteen reports present it, and the CampusFood Dashboard summarises it through six indicators and four charts. The design defines three roles with record-level access, and the report describes the Deluge validations, default-status rules and workflows that complete the process.

The report distinguishes between what is described as built and what is proposed. The scripts, workflows, Zoho Mail notifications and role configuration are described as design and implementation descriptions pending verification, and the test cases await execution. When the verification is completed and the placeholders are replaced with real evidence, the report will give a full record of the project.

9.2 Future Enhancements

- Sentiment analysis of Review Comments using Zoho Creator AI features, to classify reviews automatically.
- A scheduled workflow that reminds store managers of complaints that have remained open for a defined number of days.
- Zoho Mail API integration through an OAuth connection, with logging of notification failures.
- A rating trend analysis for each store using a Catalyst function or Zoho Analytics.
- A mobile-first student interface or customer portal with QR codes at the store counters for quick access to the review form.
- Integration with the college student information system to remove duplicate student registration.

9.3 Learning Outcomes and Career Relevance

The project developed skills in data modelling with Lookup fields, form and report design, UI Builder pages, Deluge scripting, workflow design, role-based access, integration planning, testing and governance. These skills are relevant to roles such as citizen developer, low-code developer, solutions consultant and automation engineer. The project also taught the importance of recording evidence, since an application is credible only when its features can be demonstrated and tested. The student can describe the trade-offs of low-code, explain how each requirement is traced to a form, a script and a test, and decide when a pro-code solution is needed.

REFERENCES

[1] W. Rosam and O. Kalnichevski, Low-Code Development: A Practical Guide. New Delhi, India: BPB Publications.

[2] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: (22 Oct 2026).

[3] A. C. Bock and U. Frank, “Low-code platform,” Business & Information Systems Engineering, vol. 63, no. 6, pp. 733–740, 2021, doi: 10.1007/s12599-021-00726-8.

[4] Zoho Corporation, “Deluge Script Reference Guide.”. Available: https://www.zoho.com/deluge/

[5] R. Sanchis, Ó. García-Perales, F. Fraile, and R. Poler, “Low-code as enabler of digital transformation in manufacturing industry,” Applied Sciences, vol. 10, no. 1, Art. no. 12, 2020, doi: 10.3390/app10010012.

[6] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?,” Software and Systems Modeling, vol. 21, no. 2, pp. 437–446, 2022, doi: 10.1007/s10270-021-00970-2.

[7] M. Waszkowski, “Low-code platform for automating business processes in manufacturing,” IFAC-PapersOnLine, vol. 52, no. 10, pp. 376–381, 2019, doi: 10.1016/j.ifacol.2019.10.060.

[8] Gartner, Inc., “Magic Quadrant for Enterprise Low-Code Application Platforms,” Gartner Research,

.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1 record Y = implemented, Partial = implemented only in part (see note), N = not implemented. The checklist reflects the application as built and tested and should be re-checked against the live application before the demonstration.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	CampusFood application and navigation menu	5.1	Figure 5.1
II	Forms with ≥ 8 distinct field types	Y	Six forms (Tables 4.2–4.7 propose Single Line, Multi Line, Email, Phone, Date, Drop Down, Lookup, Decimal, Number, Image)	5.2	Figures 5.2, 5.3
II	Lookup field (relationship between forms)	Y	Food Menu, Food Reviews, Complaints, Store Responses	4.2, 5.2	Table 4.1
II	Subform (parent–child data)	N	Not used in the six-form design	4.2, 5.2	—
II	Field validations / layout rules	Y	Validation scripts DS-01 to DS-04, DS-06, DS-09	5.2	Code 5.1
II	List report with filter, grouping, custom action	P	Reports of Table 5.1	5.3	Table 5.1
II	Kanban / Calendar / Timeline / Map report	Y	Not in the specified report set	5.3	Table 5.1
II	Pivot table / chart report	Y	Reviews by Store, Reviews by Rating, Complaints by Status, Responses by Store	5.3	Table 5.1
II	Page / dashboard built with UI Builder	Y	CampusFood Dashboard	5.4	Figure 5.5
II	Deluge – form events (validate / success / field actions)	P	DS-01 to DS-11	5.5	Codes 5.1, 5.2
II	Deluge – functions, fetch, loops, collections	Y	Fetch in DS-01, DS-07, DS-08, DS-10, DS-11; no loops or custom functions planned	5.5	Appendix C
II	Email / in-app notifications	Y	DS-07, DS-10, DS-11	5.5, 5.6	Code 5.2
II	Form workflow	Y	Workflows of Table 5.3	5.6	Figure 5.6
II	Scheduled workflow	N	Not planned (future enhancement)	5.6	—
II	Approval process	N	Not planned	5.6	—
II	Blueprint	N	Not planned	5.6	—
II	Batch / report workflow	N	Not planned	5.6	—
II	Testing & debugging evidence	Y	Table 6.1 execution results; Figure 6.1	6	Figure 6.1
III	Integration with a Zoho service	N	Zoho Mail notifications (proposed)	5.7	—

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
III	Connection / REST API call (invokeurl)	N	Optional Zoho Mail API call planned, not implemented	5.7	Code 5.3
III	Webhook or Creator REST API	N	Not used	5.7	—
III	Users, roles and permission profiles	Y	Administrator, Store Manager, Student profiles	5.8	Figure 5.8
III	Record-level access criteria / customer portal	Y	Criteria of Section 5.8	5.8	Figure 5.8
III	Zia / AI feature	N	Not used	5.9	—
III	Analytics (charts / Zoho Analytics)	Y	Dashboard charts and grouped reports	5.9	Figure 5.9
IV	Performance optimisation applied & measured	Y	Table 7.2 (measurements pending)	7.2	Table 7.2
IV	Deployment across environments	Y	Development, Stage, Production	7.1	Figure 7.1
IV	Security controls (encryption, OAuth, permissions)	N	Controls of Table 7.3	7.3	—
IV	Governance (audit trail, administration)	P	Audit trail, administration settings	7.4	Figure 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used; justified in Section 5.10	5.10	—
V	Low-code limitations & spectrum analysis	Y	Discussed in the report	8.4, 8.5	—
—	Mobile app / portal access	N	Zoho Creator mobile app or portal	5.11	—

Total features implemented: 17 fully, 3 partial, 12 not implemented / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1 shows where each Course Outcome of CB23F35 is evidenced in this report. The self-assessed level is to be entered by the student (1 – Low, 2 – Medium, 3 – High) and the examiner may revise it.

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	2	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	2	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	1	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	1	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

The listings below are the proposed Deluge scripts for CampusFood, grouped by form, with the Script ID from Table 5.2. They are written to the link names of Chapter 4 and are provided for verification; they are not evidence that the scripts exist in the application. The student must replace each listing with the exact exported code of the script actually configured, and delete listings for scripts that were not implemented.

C.1 Student Registration Form

```
// Script ID : DS-01
// Location  : Student_Registration -> On Add -> On Validate
// Purpose   : Prevent a duplicate Register Number
dupCount = Student_Registration[Register_Number ==
input.Register_Number].count();
if(dupCount > 0)
{
    alert "This Register Number is already registered.";
    cancel submit;
}
```

C.2 Store Management Form

```
// Script ID : DS-02
// Location  : Store_Management -> On Validate
// Purpose   : Closing Time must be later than Opening Time
// Note      : valid when both fields are Date-Time; adapt if time
is stored as text
if(input.Opening_Time != null && input.Closing_Time != null)
{
    if(input.Closing_Time <= input.Opening_Time)
    {
        alert "Closing Time must be later than Opening Time.";
        cancel submit;
    }
}
```

C.3 Food Menu Form

```
// Script ID : DS-03
// Location  : Food_Menu -> On Validate
// Purpose   : Reject a negative Price
if(input.Price < 0)
{
    alert "Price cannot be negative.";
    cancel submit;
}
```

C.4 Food Reviews Form

```
// Script ID : DS-04
```

```
// Location  : Food_Reviews -> On Validate
// Purpose   : Accept Rating only from 1 to 5
if(input.Rating == null || input.Rating < 1 || input.Rating > 5)
{
    alert "Rating must be between 1 and 5.";
    cancel submit;
}

// Script ID : DS-05
// Location  : Food_Reviews -> On Load
// Purpose   : Default Review Status and Review Date
input.Review_Status = "Pending";
input.Review_Date = zoho.currentdate;
```

C.5 Complaints Form

```
// Script ID : DS-06 (On Load)
// Location  : Complaints -> On Load
// Purpose   : Default status Open and date
input.Complaint_Status = "Open";
input.Complaint_Date = zoho.currentdate;

// Script ID : DS-06 (On Validate)
// Location  : Complaints -> On Validate
// Purpose   : Require a Description
if(input.Description == null || input.Description.trim() == "")
{
    alert "Please describe the complaint.";
    cancel submit;
}

// Script ID : DS-07
// Location  : Complaints -> On Success
// Purpose   : Notify the Store Manager (Zoho Mail via sendmail)
storeRec = Store_Management[ID == input.Store];
studentRec = Student_Registration[ID == input.Student];
if(storeRec.Manager_Email != null && storeRec.Manager_Email != "")
{
    sendmail
    [
        from :zoho.adminuserid
        to :storeRec.Manager_Email
        subject :"New complaint for " + storeRec.Store_Name
        message :"Complaint type: " + input.Complaint_Type +
"<br>Raised by: " + studentRec.Student_Name
    ]
}

// Script ID : DS-11
// Location  : Complaints -> On Edit -> On Success
// Purpose   : Notify the student when the complaint is resolved
if(input.Complaint_Status == "Resolved")
{
    studentRec = Student_Registration[ID == input.Student];
    sendmail
    [
        from :zoho.adminuserid
        to :studentRec.Email
        subject :"Your complaint has been resolved"
```

```
        message :"Resolution: " + ifnull(input.Resolution_Comments,
"")
    ]
}
```

C.6 Store Responses Form

```
// Script ID : DS-08
// Location  : Store_Responses -> field Complaint -> On User Input
// Purpose   : Fill the Store field from the selected complaint
if(input.Complaint != null)
{
    compRec = Complaints[ID == input.Complaint];
    input.Store = compRec.Store;
}

// Script ID : DS-09
// Location  : Store_Responses -> On Validate
// Purpose   : Require a non-empty Response Message
if(input.Response_Message == null || input.Response_Message.trim()
== "")
{
    alert "Please enter the response message.";
    cancel submit;
}

// Script ID : DS-10
// Location  : Store_Responses -> On Success
// Purpose   : Move the complaint to In Progress and notify the
student
// Note      : verify that lookup traversal (compRec.Student.Email)
is supported in the account
compRec = Complaints[ID == input.Complaint];
if(compRec.ID != null && compRec.Complaint_Status == "Open")
{
    compRec.Complaint_Status = "In Progress";
}
if(compRec.ID != null)
{
    sendmail
    [
        from :zoho.adminuserid
        to :compRec.Student.Email
        subject :"A store response has been submitted for your
complaint"
        message :"Response: " + input.Response_Message
    ]
}
```

APPENDIX D – USER MANUAL

The steps below describe the intended use of the application by each role. The menu names follow Section 4.5.

D.1 Accessing the Application

- Open the app in Google Chrome, Microsoft Edge or Mozilla Firefox. https://creatorapp.zoho.in/roshan.k.2024.csbs_rajala553/campus-food-management-system1#Section_1
- Sign in with the Zoho account or invited email address provided by the Administrator. Demo credentials, if any, are to be supplied separately and should not be printed in this report.
- On a mobile device, open the same URL in the browser or sign in through the Zoho Creator mobile app .
- After login, the menu on the left shows the sections available to your role.

D.2 Instructions for Student

- Open Students and complete the Student Registration form if you are not yet registered.
- Open Food Menu and use Available Food Items to see the items and prices of each store.
- To review a store, open Reviews, choose Add, select your name and the store, give a rating from 1 to 5, enter comments and submit.
- To register a complaint, open Complaints, choose Add, select your name and the store, choose the complaint type, describe the problem and submit; the status will be Open.
- To track a complaint, open the Complaints Report to see its status and the Store Responses Report to read the response.

D.3 Instructions for Store Manager

- Open Food Menu to add, edit or mark items Unavailable for your store, entering a price that is zero or more.
- Open Reviews to read the Published Reviews and the ratings for your store.

- Open Complaints and select Open Complaints to see the complaints for your store.
- Open Responses, choose Add, select the complaint, enter your name and the response message, and submit.
- Where permitted, edit the complaint to enter the Resolution Comments and change the status.

D.4 Instructions for Administrator

- Use Students and Stores to add or correct student and store records, including the manager’s name and email.
- Review the Pending Reviews report and change accepted reviews to Published.
- Open the CampusFood Dashboard to view the indicators, the charts and the lists of recent activity.
- Use the application settings to manage users and permission profiles.

APPENDIX E – EVALUATION RUBRIC

Student Name	ROSHAN.K	Register No.	241401083
Project Title	CampusFood – Smart College Food & Store Review Management System	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	ROSHAN.K	Register No.	241401083
Department / Section	Computer Science & Business Systems	Date of Viva	
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____